Inference Engineering

The complete 102-day lecture series

Independent study edition

From token generation to a production inference stack

Runtime foundations, implementation, serving infrastructure, observability, multimodal inference, advanced techniques, and a five-day capstone.

Every day includes learning objectives, lecture notes, a worked example, a lab or implementation assignment, and a comprehension check.

This independent course follows a university lecture-and-problem-set format. It is not affiliated with Stanford University. Days 01-26 refer to the existing course notebooks; later days supply written student assignments. Numerical examples are illustrative and are not reported hardware benchmarks.

Contents

Course guide 6

Day 01 - Lecture 01: LLM Inference Mechanics: End-to-end text generation 7

Day 02 - Lecture 02: Inference from Scratch: Model internals & tokenization 8

Day 03 - Lecture 03: Embeddings: From integers to vectors 9

Day 04 - Lecture 04: Transformer Blocks & Attention Deep Dive 10

Day 05 - Lecture 05: KV Cache 11

Day 06 - Lecture 06: Ops:Byte Ratio & Arithmetic Intensity 12

Day 07 - Lecture 07: CUDA Kernels, Kernel Selection & Kernel Fusion 13

Day 08 - Lecture 08: PyTorch, Model File Formats, ONNX & TensorRT 14

Day 09 - Lecture 09: vLLM: PagedAttention & Continuous Batching 15

Day 10 - Lecture 10: SGLang: RadixAttention & Structured Outputs 16

Day 11 - Lecture 11: TensorRT-LLM: Compilation & Plugin System 17

Day 12 - Lecture 12: NVIDIA Dynamo: Disaggregated Serving 18

Day 13 - Lecture 13: Quantization: Number Formats (FP8, INT8, INT4, NVFP4) 19

Day 14 - Lecture 14: Quantization Algorithms: GPTQ, AWQ, SmoothQuant 20

Day 15 - Lecture 15: Speculative Decoding: Draft-Target, Medusa, EAGLE 21

Day 16 - Lecture 16: KV Cache: Prefix Caching & Cache-Aware Routing 22

Day 17 - Lecture 17: Model Parallelism: Tensor, Expert, Pipeline & Data 23

Day 18 - Lecture 18: Disaggregation: Prefill/Decode Split 24

Day 19 - Lecture 19: GPU Architecture: SMs, Memory Hierarchy, HBM 25

Day 20 - Lecture 20: GPU Generations: Hopper, Ada, Blackwell, Rubin 26

Day 21 - Lecture 21: Multi-GPU Instances & Multi-Instance GPU (MIG) 27

Day 22 - Lecture 22: Containerization: Docker & NVIDIA NIMs 28

Day 23 - Lecture 23: Autoscaling: Concurrency, Batching & Cold Starts 29

Day 24 - Lecture 24: Routing, Load Balancing & Queueing 30

Day 25 - Lecture 25: Multi-Cloud Capacity Management 31

Day 26 - Lecture 26: Zero-Downtime Deployment & Cost Estimation 32

Day 27 - Lecture 27: Performance Benchmarking: Tooling & Profiling 33

Day 28 - Lecture 28: Observability: Metrics, Tracing & Dashboards 34

Day 29 - Lecture 29: Client Code: Streaming, Async & Protocol Support 35

Day 30 - Lecture 30: Implement a BPE tokenizer from scratch 36

Day 31 - Lecture 31: Build a bare autoregressive decoder loop in PyTorch 37

Day 32 - Lecture 32: Implement scaled dot-product attention (SDPA) with masking 38

Day 33 - Lecture 33: Implement Flash Attention (simplified, tiling in Python) 39

Day 34 - Lecture 34: Profile attention memory growth across sequence lengths 40

Day 35 - Lecture 35: Build an INT8 quantization pipeline: quantize → dequantize → measure error 41

Day 36 - Lecture 36: Curvature-aware quantization: scale search and GPTQ distinctions 42

Day 37 - Lecture 37: Sweep quantization bit widths and plot perplexity vs compression 43

Day 38 - Lecture 38: Simulate draft-target speculative decoding with acceptance sampling 44

Day 39 - Lecture 39: Build a simple KV cache manager (block allocator, eviction policy) 45

Day 40 - Lecture 40: Implement prefix caching with hash-based deduplication 46

Day 41 - Lecture 41: Simulate tensor parallelism: split a matmul across N workers 47

Day 42 - Lecture 42: Benchmark ops:byte ratio in practice across matrix sizes 48

Day 43 - Lecture 43: CUDA profiling: profile a PyTorch model with **`torch.profiler`** 49

Day 44 - Lecture 44: Write a custom elementwise CUDA kernel via Triton 50

Day 45 - Lecture 45: Build a PyTorch custom op with CUDA backend 51

Day 46 - Lecture 46: Deploy vLLM on spark-01, benchmark TTFT and throughput 52

Day 47 - Lecture 47: Deploy SGLang, benchmark structured output latency 53

Day 48 - Lecture 48: TensorRT-LLM: compile a model and compare with eager PyTorch 54

Day 49 - Lecture 49: NVIDIA Dynamo: run a disaggregated prefill experiment 55

Day 50 - Lecture 50: Simulate continuous batching: queue arrivals, dynamic batch formation 56

Day 51 - Lecture 51: Visualize PagedAttention block layout and fragmentation 57

Day 52 - Lecture 52: Benchmark TTFT vs throughput tradeoff across batch sizes 58

Day 53 - Lecture 53: Write a production Dockerfile for a vLLM inference server 59

Day 54 - Lecture 54: NVIDIA NIM deployment and custom serving-container contracts 60

Day 55 - Lecture 55: Simulate an autoscaling policy: requests per second → replica count 61

Day 56 - Lecture 56: Measure cold start latency: model load times at different sizes 62

Day 57 - Lecture 57: Implement round-robin and least-connections load balancers 63

Day 58 - Lecture 58: Build a priority request queue with batch formation 64

Day 59 - Lecture 59: Multi-GPU tensor parallel benchmark across spark-01 and spark-02 65

Day 60 - Lecture 60: MIG capability checks and partition planning on supported GPUs 66

Day 61 - Lecture 61: GPU cost model: $/token across instance types at different utilizations 67

Day 62 - Lecture 62: Blue-green deployment: zero-downtime model version swap 68

Day 63 - Lecture 63: Emit Prometheus metrics from an inference server 69

Day 64 - Lecture 64: Build a Grafana dashboard: TTFT, TBT, queue depth, GPU utilization 70

Day 65 - Lecture 65: Add distributed tracing (OpenTelemetry) to an inference request 71

Day 66 - Lecture 66: Load test with Locust: ramp traffic, find saturation point 72

Day 67 - Lecture 67: Profile with Nsight Systems: identify kernel launch overhead 73

Day 68 - Lecture 68: Build a streaming inference client using SSE 74

Day 69 - Lecture 69: Async batch inference client using **`asyncio`** + **`aiohttp`** 75

Day 70 - Lecture 70: Multi-cloud routing: geo-aware latency-based request routing 76

Day 71 - Lecture 71: GPU memory profiling: find where your memory budget goes 77

Day 72 - Lecture 72: Benchmark quantization levels on real throughput: FP16 vs INT8 vs INT4 78

Day 73 - Lecture 73: Measure speculative decoding acceptance rates by draft model size 79

Day 74 - Lecture 74: Measure KV cache hit rates across real traffic patterns 80

Day 75 - Lecture 75: Tensor parallelism scaling: throughput and latency vs GPU count 81

Day 76 - Lecture 76: End-to-end latency breakdown: tokenization → TTFT → TBT → detokenization 82

Day 77 - Lecture 77: Build a reusable inference benchmark harness 83

Day 78 - Lecture 78: Vision Language Model (VLM) inference: image preprocessing and batching 84

Day 79 - Lecture 79: Embedding model inference: batching and throughput optimization 85

Day 80 - Lecture 80: ASR (Whisper): single-chunk and long-file latency optimization 86

Day 81 - Lecture 81: TTS: streaming real-time text-to-speech 87

Day 82 - Lecture 82: Image generation: diffusion model inference and kernel optimization 88

Day 83 - Lecture 83: Video generation: context parallelism and attention optimization 89

Day 84 - Lecture 84: Multi-modal batching: mixing text and image requests 90

Day 85 - Lecture 85: Embedding similarity search pipeline: embed → index → query 91

Day 86 - Lecture 86: Speech-to-speech pipeline: ASR → LLM → TTS end-to-end latency 92

Day 87 - Lecture 87: Long context: RoPE scaling, context parallelism across GPUs 93

Day 88 - Lecture 88: EAGLE speculative decoding: feature-level draft vs token-level 94

Day 89 - Lecture 89: Medusa: multi-head speculative decoding, measure speedup 95

Day 90 - Lecture 90: MoE routing from scratch: top-K gating, load balancing loss 96

Day 91 - Lecture 91: Expert parallelism: simulate routing across N expert shards 97

Day 92 - Lecture 92: Dynamic disaggregation with NVIDIA Dynamo 98

Day 93 - Lecture 93: Cache-aware routing: route requests to maximize KV cache hits 99

Day 94 - Lecture 94: Long context without context parallelism: chunked prefill 100

Day 95 - Lecture 95: Fine-tuning a small model for inference quality vs a large quantized one 101

Day 96 - Lecture 96: Distillation for inference: teacher-student latency/quality tradeoffs 102

Day 97 - Lecture 97: Intelligence evaluation: build an eval harness for a deployed model 103

Day 98 - Lecture 98: Design: sketch the full inference stack for a real use case 104

Day 99 - Lecture 99: Build: FastAPI + vLLM inference server with health checks and metrics 105

Day 100 - Lecture 100: Deploy: ship it to the home lab cluster with load balancing 106

Day 101 - Lecture 101: Optimize: run the benchmark harness, find the bottleneck, fix it 107

Day 102 - Lecture 102: Reflect: what I learned, what I'd do differently, what's next 108

Course guide

Prerequisites: Python, basic NumPy/PyTorch, matrix multiplication, probability, and basic command-line use. CUDA experience is useful from Day 07 onward. Review tensor shapes before starting Day 01.

Outcome: Explain the generation pipeline, derive memory and performance estimates, evaluate serving optimizations, and design an inference service with measurable latency, quality, and cost targets.

Pace: One numbered session per study day; day labels are sequence numbers rather than calendar dates. Allow 90 minutes for each taught session, plus 30-60 minutes for its assignment. Implementation projects later in the course may span several study sessions.

Coverage: All 102 days include lecture notes, worked examples, lab instructions, and problem sets. Days 01-26 link the existing notebooks. Days 27-102 provide complete written lessons and student implementation assignments; they do not include prebuilt lab notebooks or claimed GPU benchmark results. The repository's "100 Days" title is a challenge name: its existing roadmap contains 102 topics.

Daily lecture format

Time	Activity
0-10 min	Review prerequisites and state the problem
10-35 min	Develop the core model and equations
35-50 min	Work through the numerical example
50-75 min	Explore the linked notebook or simulation
75-90 min	Discuss results and attempt the problem set

Each lecture gives learning objectives, theory notes, a worked example, a notebook lab or written lab procedure, a problem set, and a comprehension check. Existing day READMEs remain companion summaries. Notebook demonstrations may use simulations or require specific hardware; identify which before interpreting results.

Measurement and submission conventions

For every lab, submit the executed notebook or clearly labeled simulation, a short interpretation, and the requested plot or table. Record model revision, device, software versions, dtype, batch size, input/output lengths, warmup, and repetition count. Separate setup time from steady-state timing. State measurement boundaries and synchronize asynchronous GPU work when timing it; consult the PyTorch benchmarking guide.

Use numerical tolerances and quality checks before claiming an optimization works. Report failed runs and unsupported features explicitly. Hypothetical examples in the lecture notes illustrate arithmetic; they are not hardware benchmarks or price quotations.

Assessment

- Daily problem sets and lab reports: 40%.
- Day 18 checkpoint: 15%. Compare an uncached baseline with two optimizations, including correctness, latency, memory, and tradeoffs.
- Day 26 systems review: 15%. Defend a capacity and deployment plan against a burst, a failed worker, and a model rollback.
- Days 98-102 capstone: 30%. Design, build, deploy in an available lab environment, evaluate, and document a serving stack.

Assess reports for correctness (40%), reproducible evidence (30%), interpretation of tradeoffs (20%), and clarity (10%). Hardware access is not required for conceptual exercises; simulated results must remain clearly distinguished from measurements.

Day 01 - Lecture 01: LLM Inference Mechanics: End-to-end text generation

Session: 90 minutes plus assignment. Preparation: Review Python arrays, matrix shapes, and conditional probability; read the companion summary.

Learning objectives

- Trace prefill and decode.
- Distinguish latency from throughput.

Lecture notes

An autoregressive model factors a sequence probability into conditional next-token probabilities. Tokenization supplies integer IDs, the forward pass produces logits, and a decoding rule chooses the next ID. Prefill processes the prompt; subsequent decode iterations extend its context. Time to first token includes more than model execution: queueing and tokenization also matter.

Board example

For a request producing 11 tokens, suppose time to first token is 100 ms and each of the remaining 10 tokens takes 20 ms. End-to-end latency is $100 + 10 \times 20 = 300$ ms. Request throughput and aggregate token throughput cannot be inferred from this single latency measurement.

Notebook lab

- llm-inference-mechanics

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Derive $p(x_1,\ldots,x_n)$ as a product of conditional probabilities. Compare greedy and temperature-based generation on three prompts. Submit a per-token latency plot and identify which work your timer includes.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 02 - Lecture 02: Inference from Scratch: Model internals & tokenization

Session: 90 minutes plus assignment. Preparation: Review Day 01 and the companion summary.

Learning objectives

- Inspect tensor shapes and parameter counts.
- Explain BPE encoding.

Lecture notes

Weights encode learned arrays; architecture code defines how those arrays are used. A tokenizer supplies a separate mapping between text and IDs. BPE training learns merge rules, while encoding applies the learned rules in priority order. The model configuration, tokenizer, and weights must agree on vocabulary and dimensions.

Board example

An embedding table with vocabulary 50,000 and hidden width 768 contains 38,400,000 parameters. At two bytes per parameter it occupies 76.8 MB in decimal units, before metadata. Tied input and output weights share storage; counting both as separate parameters would overestimate the model.

Notebook lab

- 02-a-whats-inside-a-model
- 02-b-tokenization

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Count parameters by tensor group without double-counting shared tensors. Trace the merges for one string and test encode/decode round trips on whitespace and Unicode. Explain why fewer tokens alone does not establish better task quality.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 03 - Lecture 03: Embeddings: From integers to vectors

Session: 90 minutes plus assignment. Preparation: Review Day 02 and the companion summary.

Learning objectives

- Compute embedding lookups.
- Track position and output projection shapes.

Lecture notes

An embedding layer selects rows from a learned matrix. GPT-2 adds learned position embeddings before the transformer blocks. Later architectures may represent position differently, so this example is architecture-specific. Similarity between raw token embeddings is a geometric observation, not a guarantee of semantic equivalence in context.

Board example

With vocabulary V=10,000 and width d=64, E has shape [10,000,64]. IDs [7,2,7] produce a [3,64] tensor. Rows one and three initially match; adding different position vectors can make their transformer inputs different. Projection through E^T maps a hidden vector back to 10,000 logits when weights are tied.

Notebook lab

- embeddings

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Manually gather three embedding rows and check equality with the library lookup. Compare repeated tokens before and after position addition. Report tensor shapes at each step and explain the output projection.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 04 - Lecture 04: Transformer Blocks & Attention Deep Dive

Session: 90 minutes plus assignment. Preparation: Review Day 03 and the companion summary.

Learning objectives

- Derive scaled dot-product attention.
- Apply a causal mask.

Lecture notes

Queries compare each position with keys; softmax turns the scores into weights used to combine values. Scaling by $\sqrt{d_k}$ moderates score magnitude. A causal mask excludes future positions before softmax. Residual connections, normalization, and feed-forward layers complete the block; attention alone is not the entire transformer.

Board example

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k} + M)V$. For three tokens, the allowed key positions are {1}, {1,2}, and {1,2,3}. A first-position row of scores [2,1,0] becomes [2,-∞,-∞] and therefore weights [1,0,0].

Notebook lab

- attention

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Compute a two-token attention example by hand. Verify row sums and masked probabilities numerically. Change a future token and check that an earlier output stays unchanged in deterministic evaluation mode.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 05 - Lecture 05: KV Cache

Session: 90 minutes plus assignment. Preparation: Review Day 04 and the companion summary.

Learning objectives

- Implement incremental attention.
- Calculate KV-cache storage.

Lecture notes

Cached decoding stores past keys and values so they do not need to be projected again. Each new query still attends to the preceding context; caching does not make attention independent of context length. Cache positions, masks, and layer identity must remain consistent with the uncached computation.

Board example

KV bytes = 2 × layers × batch × sequence length × KV heads × head dimension × bytes per element. With 12 layers, batch 1, length 1,024, 12 KV heads, head dimension 64, and FP16 storage, the result is 37,748,736 bytes = 36 MiB. Grouped-query attention uses KV-head count here, not query-head count.

Notebook lab

- kv-cache

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Compare cached and uncached logits within a stated tolerance. Sweep context length and plot latency plus allocated cache bytes. Explain both the memory increase and the computation avoided.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 06 - Lecture 06: Ops:Byte Ratio & Arithmetic Intensity

Session: 90 minutes plus assignment. Preparation: Review Day 05 and the companion summary.

Learning objectives

- Calculate arithmetic intensity.
- Use a roofline bound.

Lecture notes

Arithmetic intensity divides operations by bytes moved at a specified memory boundary. A roofline model bounds achievable throughput by the lower of compute capacity and bandwidth times intensity. It is an upper bound, not a prediction that accounts for every launch, dependency, or communication cost.

Board example

For hypothetical bandwidth 1 TB/s, compute ceiling 100 TFLOP/s, and intensity 10 FLOP/byte, the bandwidth ceiling is 10 TFLOP/s. The ridge point is 100 FLOP/byte. Raising intensity from 10 to 20 can improve the bound without changing peak compute.

Notebook lab

- ops-byte-ratio

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Estimate FLOPs and transferred bytes for one matrix multiplication. Sweep batch size and compare measured performance with the bound. State whether you count weight, activation, and output traffic.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 07 - Lecture 07: CUDA Kernels, Kernel Selection & Kernel Fusion

Session: 90 minutes plus assignment. Preparation: Review Day 06 and the companion summary.

Learning objectives

- Explain kernel dispatch.
- Evaluate fusion against a baseline.

Lecture notes

A kernel launches parallel work on the GPU. Separate elementwise operations can materialize intermediate arrays and incur multiple launches. Fusion can reduce those costs, but register pressure and resource use may offset the benefit. Shape, layout, and dtype influence which implementation performs well.

Board example

For y=ReLU(ax+b), a simplified three-pass implementation reads and writes an N-element array three times: about 6N element transfers. One fused pass can approach 2N transfers, excluding scalar loads. This traffic ratio is not automatically the wall-clock speedup.

Notebook lab

- cuda-kernels

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Profile an elementwise chain before and after compilation or fusion. Warm up both paths, verify outputs, and report launches, memory traffic assumptions, and elapsed time for small and large arrays.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 08 - Lecture 08: PyTorch, Model File Formats, ONNX & TensorRT

Session: 90 minutes plus assignment. Preparation: Review Day 07 and the companion summary.

Learning objectives

- Distinguish weights, graphs, and engines.
- Separate compile cost from execution.

Lecture notes

A weight container stores tensors. A graph representation describes operations, and an optimized engine specializes execution for supported hardware and shapes. Exporting a model does not guarantee that every operation or dynamic shape works on the destination runtime. Validate numerical behavior after each conversion.

Board example

Suppose compilation costs 30 seconds, eager execution costs 20 ms per call, and compiled execution costs 10 ms. Break-even is 30/(0.020-0.010)=3,000 calls, ignoring other costs. A short job can lose overall time despite faster steady-state execution.

Notebook lab

- pytorch-model-formats

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Inspect one saved weight file and compare eager with compiled execution where supported. Report setup time separately from warm latency. Test at least two input shapes and record conversion or graph-break limitations.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 09 - Lecture 09: vLLM: PagedAttention & Continuous Batching

Session: 90 minutes plus assignment. Preparation: Review Day 08 and the companion summary.

Learning objectives

- Explain block-based kv allocation.
- Simulate continuous batching.

Lecture notes

Paged KV storage maps logical token blocks onto physical memory blocks. Continuous batching admits new requests as others finish, improving utilization when lengths differ. Memory allocation and scheduling are related but distinct mechanisms: one reduces wasted capacity while the other changes when requests execute.

Board example

With block size 16 and lengths 17 and 31, the requests each need two blocks. There are 64 allocated token slots and 48 occupied slots, leaving 16 unused. A static batch with output lengths 2 and 8 cannot replace the short request unless its scheduler permits new admission.

Notebook lab

- vllm-paged-attention

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Simulate block allocation for mixed lengths and compare waste with a fixed reservation per request. Trace a scheduler timeline with arrivals and completions. Report useful tokens per second alongside time-to-first-token percentiles.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 10 - Lecture 10: SGLang: RadixAttention & Structured Outputs

Session: 90 minutes plus assignment. Preparation: Review Day 09 and the companion summary.

Learning objectives

- Trace prefix-tree reuse.
- Distinguish output syntax from correctness.

Lecture notes

A radix tree represents shared token prefixes compactly. Reusing a compatible prefix can avoid repeated prefill computation. Constrained decoding narrows valid token choices according to a grammar or schema, but syntactic validity does not establish factual accuracy or useful task completion.

Board example

Requests with token sequences [1,2,3,4] and [1,2,3,9] share a three-token prefix. A cache hit can reuse those prefix states when execution configuration matches. A valid JSON object containing an incorrect answer still fails a factual evaluation.

Notebook lab

- sglang-radix-attention

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Construct a small prefix tree and measure reusable tokens for repeated and unique prompts. Evaluate structured outputs using both schema validation and a task-specific correctness check. Report these metrics separately.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 11 - Lecture 11: TensorRT-LLM: Compilation & Plugin System

Session: 90 minutes plus assignment. Preparation: Review Day 10 and the companion summary.

Learning objectives

- Explain engine specialization.
- Evaluate build and runtime tradeoffs.

Lecture notes

An optimized inference runtime can select specialized implementations and execution plans for supported workloads. Plugins supply operations that need custom handling. Optimization requires an explicit support envelope: model architecture, precision, shapes, device, and runtime version all affect whether an execution path is usable.

Board example

If one execution plan covers sequence lengths up to 2,048, a 4,096-token request needs an explicitly supported alternative. Silently truncating it changes the task. If a build costs 120 seconds and saves 5 ms per call, amortization requires 24,000 calls.

Notebook lab

- tensorrt-llm

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Describe a supported model/shape/precision configuration using the notebook. Compare build cost and warm execution where hardware permits; otherwise use its simulation and label results as simulated. Specify behavior for unsupported requests.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 12 - Lecture 12: NVIDIA Dynamo: Disaggregated Serving

Session: 90 minutes plus assignment. Preparation: Review Day 11 and the companion summary.

Learning objectives

- Trace disaggregated request flow.
- Estimate KV-transfer cost.

Lecture notes

Disaggregated serving assigns prompt processing and token generation to separate worker pools. The decoder needs the correct KV state before continuing the request. Routing, transfer, admission, and failure handling are part of the design; extra communication can consume the benefit of independent scaling.

Board example

For a 512 MiB cache and effective transfer bandwidth 25 GiB/s, the ideal transfer time is $0.5/25=0.02$ seconds, or 20 ms. Allocation, serialization, coordination, and contention add to that bound. A 10 ms compute saving cannot repay even the ideal transfer cost in this example.

Notebook lab

- nvidia-dynamo

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Trace one request from admission through prefill, transfer, and decode. Sweep cache size and effective bandwidth in the notebook model. Explain what happens if the prefill worker fails before transfer completes.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 13 - Lecture 13: Quantization: Number Formats (FP8, INT8, INT4, NVFP4)

Session: 90 minutes plus assignment. Preparation: Review Day 12 and the companion summary.

Learning objectives

- Calculate packed weight storage.
- Quantify rounding and clipping error.

Lecture notes

Quantization maps values to a smaller representable set. For affine integer quantization, q=clip(round(x/s)+z), and reconstruction is $\hat{x}$=s(q-z). Floating-point and block-scaled formats have different representations. Storage savings depend on metadata and packing; speed also depends on available kernels and workload.

Board example

A model with 7 billion parameters has 14 GB of raw two-byte weights and 3.5 GB of ideally packed four-bit weights. Scales, padding, and non-quantized tensors add overhead. With s=0.1 and z=0, x=0.26 rounds to q=3 and reconstructs to 0.3, an absolute error of 0.04.

Notebook lab

- quantization-number-formats

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Quantize a tensor at two precisions and report mean and maximum reconstruction error, clipping count, and actual stored bytes. Explain why a fourfold weight reduction does not guarantee fourfold speedup.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 14 - Lecture 14: Quantization Algorithms: GPTQ, AWQ, SmoothQuant

Session: 90 minutes plus assignment. Preparation: Review Day 13 and the companion summary.

Learning objectives

- Compare calibration objectives.
- Evaluate output error on held-out inputs.

Lecture notes

Weight-only and weight-and-activation quantization address different bottlenecks. GPTQ-style methods use curvature information to guide weight approximation; AWQ uses activation information to guide scaling; SmoothQuant redistributes scaling between activations and weights. A simplified demonstration is not a complete implementation of these algorithms.

Board example

For row-vector activations X and weights W, introducing invertible diagonal D gives $XW=(XD^{-1})(DW)$. The unquantized product is unchanged, but the ranges of the two factors change. After quantization, their errors can differ greatly, which makes calibration and evaluation necessary.

Notebook lab

- quantization-gptq-awq-smoothquant

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Verify the scaling identity numerically. Compare a rounding baseline with the notebook demonstrations on separate calibration and evaluation inputs. Report output error and describe exactly which algorithmic steps your implementation omits.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 15 - Lecture 15: Speculative Decoding: Draft-Target, Medusa, EAGLE

Session: 90 minutes plus assignment. Preparation: Review Day 14 and the companion summary.

Learning objectives

- Explain draft-and-verify decoding.
- Evaluate acceptance and overhead.

Lecture notes

A draft model proposes tokens and the target verifies them in parallel. Exact speculative sampling uses acceptance and correction rules to preserve the target distribution; simply accepting similar-looking tokens does not provide that guarantee. Benefit depends on accepted progress relative to drafting and verification costs.

Board example

Suppose a cycle costs 4 ms of drafting and 12 ms of verification and yields an average of three committed tokens. Its effective time is $16/3 \approx 5.33$ ms per token. Against an 8 ms target-only baseline, speedup is $1.5\times$. At only one committed token per cycle, it is slower than the baseline.

Notebook lab

- speculative-decoding

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Sweep draft length and measure committed tokens per cycle, total latency, and quality. Explain acceptance and rejection for the notebook method. Separate greedy verification from claims about exact stochastic sampling.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 16 - Lecture 16: KV Cache: Prefix Caching & Cache-Aware Routing

Session: 90 minutes plus assignment. Preparation: Review Day 15 and the companion summary.

Learning objectives

- Define valid prefix reuse.
- Balance cache locality against queueing.

Lecture notes

Prefix reuse requires matching token history and compatible model execution state. Cache identity should account for model version and relevant adapters or configuration. A route with a cache hit may still be slow if its queue is long. Cache eviction and invalidation are therefore part of request scheduling.

Board example

Worker A has a reusable prefix saving 40 ms but a 100 ms queue. Worker B has no hit and a 10 ms queue. Holding other costs equal, A adds 100-40=60 ms relative to uncached compute, while B adds 10 ms. Cache locality alone would select the slower worker.

Notebook lab

- kv-cache-prefix-caching

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Compare round-robin with a policy that considers both cache savings and waiting time. Include repeated prompts, unique prompts, and a model-version change. Explain how stale cache reuse is prevented.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 17 - Lecture 17: Model Parallelism: Tensor, Expert, Pipeline & Data

Session: 90 minutes plus assignment. Preparation: Review Day 16 and the companion summary.

Learning objectives

- Distinguish tensor, pipeline, expert, and data parallelism.
- Model communication.

Lecture notes

Tensor parallelism partitions tensor operations; pipeline parallelism places layer groups on different workers; expert parallelism distributes mixture-of-experts computation; data parallelism replicates the model across independent requests. Splitting computation introduces communication and coordination, so ideal scaling is an upper bound.

Board example

If 90% of execution scales perfectly across four devices and 10% remains serial, Amdahl's bound is $1/(0.1+0.9/4)\approx3.08\times$. Communication lowers this further. Four full replicas can increase serving capacity without lowering the latency of an individual request.

Notebook lab

- model-parallelism

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Choose a parallelism strategy for a model that exceeds one device's memory and another that fits. Estimate per-device memory and identify communications. Explain how expert imbalance or pipeline bubbles affect utilization.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 18 - Lecture 18: Disaggregation: Prefill/Decode Split

Session: 90 minutes plus assignment. Preparation: Review Day 17 and the companion summary.

Learning objectives

- Compare colocated and split serving.
- Locate the crossover point.

Lecture notes

Prefill and decode have different resource profiles, but these vary with batch size, context length, and hardware. Disaggregation helps when better scheduling and resource allocation outweigh transfer and coordination. Evaluate time to first token and inter-token latency separately; average throughput can hide an interactive regression.

Board example

A colocated configuration has 80 ms of contention overhead. Splitting removes 60 ms but adds 25 ms transfer and 10 ms coordination. Net improvement is 25 ms. If transfer grows to 70 ms, the same design becomes 20 ms slower.

Notebook lab

- disaggregation-prefill-decode

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Build a crossover plot over prompt length and effective network bandwidth. Compare colocated and split policies at equal total resources. State the workload region where your result changes sign.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 19 - Lecture 19: GPU Architecture: SMs, Memory Hierarchy, HBM

Session: 90 minutes plus assignment. Preparation: Review Day 18 and the companion summary.

Learning objectives

- Trace the memory hierarchy.
- Distinguish capacity, bandwidth, and latency.

Lecture notes

GPU execution distributes work across streaming multiprocessors. Registers and shared memory support local reuse, while larger memory levels serve broader access. Coalescing and locality change how much useful data is delivered. Occupancy can help hide latency but is not an optimization objective independent of actual throughput.

Board example

Reading 1 GiB over a hypothetical sustained 500 GiB/s path takes at least 2 ms. Reading the same array twice doubles traffic if no cache reuse occurs. A kernel may still take longer because of dependencies, launch overhead, or inefficient accesses.

Notebook lab

- gpu-architecture-sms-hbm

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Profile contiguous and strided accesses at multiple sizes. Compare measured bandwidth and explain whether the working set fits a cache. Identify the memory system of the actual device instead of assuming every GPU uses HBM.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 20 - Lecture 20: GPU Generations: Hopper, Ada, Blackwell, Rubin

Session: 90 minutes plus assignment. Preparation: Review Day 19 and the companion summary.

Learning objectives

- Compare hardware using workload constraints.
- Separate projections from measurements.

Lecture notes

A device comparison must include usable memory, bandwidth, supported numerical formats, interconnect, software support, and power or acquisition cost. Peak arithmetic rates describe a particular operation and precision. They cannot establish end-to-end performance for an arbitrary model.

Board example

Suppose device A sustains 1 TB/s and device B sustains 2 TB/s, while B has four times the peak compute. A purely weight-bandwidth-bound workload has an ideal 2× bandwidth-based improvement, not 4×. Real gains require measurement with matching model, precision, and workload.

Notebook lab

- gpu-generations-hopper-blackwell

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Create a dated comparison using official specifications for the devices you can access. Label notebook forecasts and hypothetical values explicitly. Predict the bottleneck and test one prediction; do not present projected generations as measured hardware.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 21 - Lecture 21: Multi-GPU Instances & Multi-Instance GPU (MIG)

Session: 90 minutes plus assignment. Preparation: Review Day 20 and the companion summary.

Learning objectives

- Distinguish partitioning from sharding.
- Assess isolation and interconnects.

Lecture notes

Multi-device execution connects separate accelerators. Hardware partitioning divides a supported device into isolated instances with constrained resources. These solve different problems. Supported partition profiles and interconnect capabilities are device-specific; a lab plan must first establish which features exist on the available hardware.

Board example

A service requiring 18 GiB for weights plus 8 GiB for cache and workspace needs at least 26 GiB before additional headroom. It cannot fit a partition with only 20 GiB even if the parent device has ample memory. Two partitions do not automatically combine into one larger allocation.

Notebook lab

- multi-gpu-instances-mig

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Inventory device capability and available memory. Design placements for two differently sized services. If hardware partitioning is unsupported, simulate placement and clearly report that no partition was created.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 22 - Lecture 22: Containerization: Docker & NVIDIA NIMs

Session: 90 minutes plus assignment. Preparation: Review Day 21 and the companion summary.

Learning objectives

- Separate image, model, and runtime configuration.
- Define readiness.

Lecture notes

An inference container packages application dependencies, while model artifacts may be bundled or supplied separately. A running process is not necessarily ready to serve: weights must load and the request path must work. Reproducibility requires recording the image digest, model revision, runtime version, and configuration.

Board example

A container starts its process after 5 seconds but finishes loading weights after 60 seconds. Routing traffic at process start exposes a 55-second failure window. A readiness probe should gate admission until the serving path is usable; liveness answers a different question.

Notebook lab

- containerization-docker-nim

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Review the notebook container example and document image/model boundaries. Define readiness, liveness, and a minimal inference smoke check. Measure image download, model loading, and warmup as separate startup stages if execution is available.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 23 - Lecture 23: Autoscaling: Concurrency, Batching & Cold Starts

Session: 90 minutes plus assignment. Preparation: Review Day 22 and the companion summary.

Learning objectives

- Relate arrival rate to concurrency.
- Account for provisioning lag.

Lecture notes

Autoscaling reacts to observed or predicted demand, but replicas need time to become useful. Queue length, in-flight work, and service-time distributions often convey more than utilization alone. Hysteresis and cooldowns reduce oscillation; capacity targets also need headroom for bursts and long requests.

Board example

Little's law gives $L=\lambda W$ in a stable system with consistent boundaries. At 10 requests/s and mean residence time 2 seconds, mean in-system concurrency is 20. If demand jumps to 20 requests/s while capacity remains 10 requests/s for 30 seconds, backlog grows by roughly 300 requests under this fluid approximation.

Notebook lab

- autoscaling-concurrency-cold-starts

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Simulate a burst with nonzero cold-start delay. Compare a reactive policy with a warm reserve. Plot queue length, active replicas, and latency; explain the cost of the reserve and the risk of scaling down too early.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 24 - Lecture 24: Routing, Load Balancing & Queueing

Session: 90 minutes plus assignment. Preparation: Review Day 23 and the companion summary.

Learning objectives

- Compare work-aware routing policies.
- Analyze fairness and stability.

Lecture notes

Request count is a weak proxy for remaining work when prompt and output lengths vary. Routing can consider queue estimates, prefix reuse, or adapter locality. Priority policies need an explicit starvation response. Queueing models are useful only when their arrival, service, and stationarity assumptions are stated.

Board example

Worker A has two requests with estimated remaining work 1 second each; B has one with 10 seconds remaining. Least-connections selects B, while least-estimated-work selects A. Estimates may be wrong, so compare policies under prediction error rather than assuming perfect information.

Notebook lab

- routing-load-balancing-queueing

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Replay mixed short and long jobs through FIFO and a work-aware policy. Report p50/p95 latency for each class and maximum wait. Inject estimation errors and show whether the policy still improves the result.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 25 - Lecture 25: Multi-Cloud Capacity Management

Session: 90 minutes plus assignment. Preparation: Review Day 24 and the companion summary.

Learning objectives

- Plan capacity across failure domains.
- Account for network and recovery costs.

Lecture notes

Multiple providers can offer alternative capacity, but failover requires more than a routing rule. Compatible serving images, accessible model artifacts, data placement, quotas, and warm capacity affect recovery. Geographic placement also changes request latency and transfer costs.

Board example

Two regions each sustain 60 requests/s and jointly serve 80. Losing one leaves only 60 requests/s, a deficit of 20. Over a 30-second recovery interval, roughly 600 requests accumulate unless traffic is shed or spare capacity becomes available.

Notebook lab

- multi-cloud-capacity

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Design a two-region capacity plan with a stated recovery target. Simulate loss of one region and include startup delay. Compare warm redundancy with cold recovery, documenting transfer costs as assumptions rather than live price quotes.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 26 - Lecture 26: Zero-Downtime Deployment & Cost Estimation

Session: 90 minutes plus assignment. Preparation: Review Day 25 and the companion summary.

Learning objectives

- Design rollout gates and draining.
- Calculate cost per useful token.

Lecture notes

A rollout must validate readiness, preserve in-flight requests where possible, and define rollback behavior. Canary evaluation needs both system metrics and task-quality checks. Cost comparisons must use consistent accounting: measured throughput already reflects observed utilization and should not be discounted a second time.

Board example

For a fleet costing $12/hour and delivering 1,000 useful tokens/s averaged over that hour, cost per million tokens is $12\times10^6/(1{,}000\times3{,}600)\approx\$3.33$. Include the entire fleet and define whether the denominator counts input, output, or both. A second identical fleet during rollout increases temporary cost.

Notebook lab

- zero-downtime-deployment

Read the notebook's setup and assumptions, then use its relevant demonstrations to investigate the assignment below. Record the baseline before changing one variable. Label simulated and measured results separately; use the course measurement conventions.

Problem set and deliverable

Specify a canary schedule, readiness gate, rollback thresholds, and drain timeout. Simulate long-lived requests during cutover. Calculate cost from observed delivered tokens and identify any excluded infrastructure expenses.

Check your understanding

Explain the worked example without looking at the notes. Name one assumption that would change its conclusion and describe an experiment that could test it.

Day 27 - Lecture 27: Performance Benchmarking: Tooling & Profiling

Session: 90 minutes plus an implementation assignment. Preparation: Day 06, Day 07, Day 26.

Learning objectives

- Define a reproducible workload.
- Distinguish device time from user-visible latency.

Lecture notes

A benchmark answers a specific question under a specified workload. Fix prompt lengths, generated lengths, arrival process, model revision, precision, and stopping rules before comparing implementations. Warmup changes the experiment by excluding initialization; keep a separate cold-start measurement. GPU execution is asynchronous, so a host timer around a launch can measure submission rather than completion.

Latency percentiles describe a distribution, while aggregate throughput counts completed work over a common interval. A closed-loop client waits for responses before issuing more work and can hide overload by reducing its own arrival rate. An open-loop generator schedules arrivals independently and must retain failures and unfinished requests in its report. Use profiling to explain a measured effect, then repeat performance measurements without heavy tracing.

Board example

For sorted request latencies [10,11,12,13,100] ms, the mean is 29.2 ms and the median is 12 ms. Under the nearest-rank convention, p95 is 100 ms. State your percentile convention and sample count; five requests cannot establish a stable production tail.

Lab procedure

Create a fixed 20-prompt fixture with short and long groups. Run warmup separately, collect at least 100 measured requests per configuration, and record timestamps and outcomes. Compare one configuration change using repeated runs; preserve timeouts rather than dropping them.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the workload manifest, raw timings, percentile table, and one profiler hypothesis.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why a lower median with more failed requests might be a regression.

Further reading

- PyTorch benchmarking

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 28 - Lecture 28: Observability: Metrics, Tracing & Dashboards

Session: 90 minutes plus an implementation assignment. Preparation: Day 24, Day 27.

Learning objectives

- Choose useful metrics and spans.
- Diagnose an inference slowdown.

Lecture notes

Metrics summarize behavior over time, traces show the stages of individual requests, and logs record discrete events. Use each for a question it can answer: a queue gauge shows backlog, a duration histogram shows distribution, and a trace locates waiting within a request. Counters must increase monotonically between restarts; gauges may rise or fall.

Observe arrival rate, completions, failures, queue depth, active requests, and stage durations together. A GPU can be busy while user-visible service deteriorates. Keep metric labels bounded: request IDs and full prompts belong neither in label sets nor routine telemetry. Histograms allow aggregation across replicas when bucket boundaries agree; averaging replica percentiles does not yield a fleet percentile.

Board example

Two workers report p95 values of 20 ms and 200 ms but process 1,000 and 10 requests respectively. Their average, 110 ms, is not the fleet p95. Merge compatible histogram counts or raw observations before calculating the fleet statistic.

Lab procedure

Instrument a simulated queue with arrivals, admissions, first-token events, completions, and failures. Introduce a slower service stage and draw the resulting queue-depth and latency curves on the same time axis.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a signal-to-question table and three incident diagnoses.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Identify which observation distinguishes slow model execution from slow admission.

Further reading

- Prometheus instrumentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 29 - Lecture 29: Client Code: Streaming, Async & Protocol Support

Session: 90 minutes plus an implementation assignment. Preparation: Day 01, Day 24, Day 28.

Learning objectives

- Implement streaming lifecycle handling.
- Bound asynchronous request concurrency.

Lecture notes

A streaming client receives incremental events, not necessarily one token per network read. Network chunks may split an event or contain several events. Parsing must retain incomplete data, decode text incrementally, and distinguish ordinary data from terminal or error events. Time to first byte, first event, and first generated token are different measurements.

Client concurrency can saturate connection pools, server queues, or memory. Set an explicit bound and timeouts for connection, inactivity, and overall request duration. Cancellation should release the connection and notify the server when supported. Retrying after partial output can repeat computation and duplicate text; make that behavior explicit rather than silently appending a new generation.

Board example

At concurrency 8 and average request time 2 seconds, an ideal closed-loop client completes about 4 requests/s. Raising concurrency to 80 does not imply 40 requests/s if the server is already saturated; it may mainly lengthen queues.

Lab procedure

Build a mock stream that splits a multibyte character and an event across reads. Exercise success, timeout, disconnect, and cancellation. Then run ten mock requests through a concurrency limit of three and record peak in-flight work.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the parser state diagram and lifecycle tests.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which errors can be retried before output and which require exposing an interrupted result.

Further reading

- Server-sent events
- aiohttp client guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 30 - Lecture 30: Implement a BPE tokenizer from scratch

Session: 90 minutes plus an implementation assignment. Preparation: Day 02, Day 29.

Learning objectives

- Train a deterministic toy BPE vocabulary.
- Distinguish training from encoding.

Lecture notes

Start from a defined alphabet and a corpus represented as symbol sequences. Count adjacent pairs within each training sequence, select the most frequent pair, merge its non-overlapping occurrences, and repeat until reaching a merge budget. Deterministic tie-breaking makes training reproducible. Boundaries and preprocessing are part of the tokenizer definition.

Encoding uses the learned merge ranks, not fresh corpus counts. A toy character tokenizer demonstrates the algorithm but is not automatically compatible with GPT-2's byte mapping, preprocessing, or special tokens. Decoding should invert the symbol mapping for supported input. Unknown-symbol behavior and Unicode handling need explicit tests rather than assumptions.

Board example

For sequences [a,b,a,b] and [a,b], pair (a,b) occurs three times. Merging it into X gives [X,X] and [X]. One merge rule reduces the corpus from six symbols to three. Subsequent encoding must apply this learned rule consistently.

Lab procedure

Implement pair counting, stable tie-breaking, non-overlapping merging, encoding, and decoding in a small Python module. Test empty input, repeated symbols, overlapping pairs such as aaa, and an unseen character. Compare your trace with the Day 02 notebook.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit merge rules, round-trip cases, and corpus token counts before and after training.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why token compression alone does not measure language-model quality.

Further reading

- Tokenization pipeline

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 31 - Lecture 31: Build a bare autoregressive decoder loop in PyTorch

Session: 90 minutes plus an implementation assignment. Preparation: Day 01, Day 04, Day 05.

Learning objectives

- Write an explicit decode loop.
- Preserve stopping and cache semantics.

Lecture notes

Separate tokenization, model execution, token selection, and stopping into inspectable steps. A minimal greedy decoder takes the final-position logits, selects their argmax, appends that ID, and repeats until EOS or a generation budget. Use evaluation mode and an inference context; dropout or gradient tracking changes the experiment.

An uncached implementation provides a correctness reference. The cached variant initially processes the prompt and then only the new token, passing the returned cache forward. Attention masks and position indices must reflect the entire logical sequence even when the latest input contains one token. Batch padding and per-sequence completion require more bookkeeping than a single-prompt loop.

Board example

If a prompt has 5 tokens and generation is capped at 3, the uncached forward input lengths are 5, 6, and 7. With a correctly initialized cache, they are 5, 1, and 1. Both should select the same greedy IDs within the effects of numerical precision.

Lab procedure

Use a small local model to implement the uncached path first. Add a cached path and compare logits and token IDs on three prompts. Exercise immediate EOS, maximum-length stopping, and an empty-input policy.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit both traces and a latency comparison with warmup excluded.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why changing sampling RNG consumption can prevent token-for-token agreement even when distributions match.

Further reading

- PyTorch inference and CUDA semantics

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 32 - Lecture 32: Implement scaled dot-product attention (SDPA) with masking

Session: 90 minutes plus an implementation assignment. Preparation: Day 04, Day 31.

Learning objectives

- Implement a numerically stable attention reference.
- Test mask alignment.

Lecture notes

Compute scores $QK^T/\sqrt{d}$, apply the causal and padding masks, subtract each row's maximum, exponentiate, and normalize before multiplying by V. For full-sequence self-attention, the causal mask is lower triangular. Cached decoding requires absolute position alignment; a one-row local triangle is not a sufficient specification of which past keys are visible.

Define behavior for fully masked rows before coding. Softmax of a row consisting entirely of negative infinity produces undefined arithmetic in a naive implementation. Padding outputs can be zeroed under an explicit policy. Keep dropout disabled in the reference, and compare both outputs and shapes with a library implementation under the same mask convention.

Board example

For scores [0, ln(3)], softmax is [1/4,3/4]. With values [2,6], the weighted result is 5. If the second key is masked, the result becomes 2. Adding the same finite constant to both unmasked scores leaves the probabilities unchanged.

Lab procedure

Implement the reference in NumPy or PyTorch. Test full attention, causal attention, padding, cached-query offsets, and a fully masked row. Compare against hand calculations before using larger random tensors.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit numerical error and mask tests.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Demonstrate that modifying a future value cannot affect an earlier causal output.

Further reading

- PyTorch scaled dot-product attention

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 33 - Lecture 33: Implement Flash Attention (simplified, tiling in Python)

Session: 90 minutes plus an implementation assignment. Preparation: Day 06, Day 32.

Learning objectives

- Derive an online softmax update.
- Avoid materializing the full score matrix.

Lecture notes

Tiling processes score blocks while maintaining per-query running statistics. Store a running maximum m, exponential sum l, and unnormalized weighted-value sum u. For a new block with scores s and values V, set $m' = \max(m, \max(s))$, rescale old l and u by $\exp(m - m')$, and add $\exp(s - m')$ and its value-weighted sum. The final output is u/l.

This is exact attention algebra up to floating-point roundoff; it does not approximate attention by dropping keys. A Python implementation teaches the recurrence but cannot claim the GPU memory traffic or speed of a fused FlashAttention kernel. Apply causal masks within each tile, and skip fully masked tiles so the initial negative-infinity state does not create NaNs.

Board example

Two scalar score blocks [0] and [ln(3)] with values [2] and [6] give $m = \ln(3)$, $l = 1/3 + 1 = 4/3$, and $u = 2/3 + 6 = 20/3$. Dividing gives 5, matching the dense calculation from Day 32.

Lab procedure

Implement tiled forward attention with running statistics. Compare several tile sizes against the Day 32 reference, including extreme logits and a non-divisible sequence length. Track the largest temporary score tensor.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the update derivation, correctness results, and temporary-memory estimate.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why a slower Python implementation can still demonstrate a better memory algorithm.

Further reading

- FlashAttention paper

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 34 - Lecture 34: Profile attention memory growth across sequence lengths

Session: 90 minutes plus an implementation assignment. Preparation: Day 05, Day 33.

Learning objectives

- Separate quadratic scores from linear KV storage.
- Measure peak memory correctly.

Lecture notes

Dense score materialization uses B×H×S×S elements, while KV storage grows with B×L×S×Hkv×d. These are different tensors with different lifetimes. An efficient attention backend can avoid a full score matrix but still needs keys and values. Model weights add a largely sequence-independent baseline.

Measure allocated and reserved memory separately, reset peak statistics for each run, and release tensors between configurations. Backend selection, dtype, and gradients affect results. A memory slope is meaningful only when those conditions remain fixed. Unified-memory systems require additional care because physical memory is shared and device accounting may differ from a discrete GPU.

Board example

For batch 1, 8 heads, S=2,048, and two-byte scores, the score tensor alone is 64 MiB. Doubling S produces 256 MiB. A fixed-architecture KV cache instead doubles when sequence length doubles.

Lab procedure

Sweep sequence lengths with the dense reference and tiled implementation. Record predicted score bytes, peak allocation, reserved memory, and selected backend. If no GPU is available, report analytical tensor sizes and CPU measurements separately.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a log-log plot and explain any discontinuities.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Identify which measurements include weights, temporary buffers, and allocator reserves.

Further reading

- PyTorch CUDA memory management

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 35 - Lecture 35: Build an INT8 quantization pipeline: quantize → dequantize → measure error

Session: 90 minutes plus an implementation assignment. Preparation: Day 13, Day 32.

Learning objectives

- Implement symmetric quantize/dequantize.
- Diagnose clipping and scale granularity.

Lecture notes

For a symmetric signed INT8 teaching scheme, choose s=max(abs(x))/127, round x/s, and clamp to [-127,127]. Reconstruct with $\hat{x}$=sq. Use a defined scale such as 1 for an all-zero group. This convention deliberately leaves -128 unused and must match the chosen implementation.

Per-tensor scaling shares one scale across all values; per-channel scaling can preserve small channels in the presence of outliers. Reconstruction error is only a local proxy: downstream output error also depends on activation directions. Storing integer weights and dequantizing before an ordinary floating-point operation demonstrates compression error, not accelerated INT8 inference.

Board example

With maximum magnitude 12.7, s=0.1. Values [-0.26,0,0.24] quantize to [-3,0,2] and reconstruct to [-0.3,0,0.2]. The maximum absolute error is 0.04. A zero tensor should reconstruct exactly without division by zero.

Lab procedure

Implement both scale granularities, then inject one large outlier. Measure storage including scales, mean-square error, maximum error, and the error of a small linear layer on held-out inputs.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the pipeline and a granularity comparison.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why dtype storage and kernel arithmetic are separate claims.

Further reading

- GPTQ: quantization and reconstruction objectives

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 36 - Lecture 36: Curvature-aware quantization: scale search and GPTQ distinctions

Session: 90 minutes plus an implementation assignment. Preparation: Day 14, Day 35.

Learning objectives

- Derive an activation-weighted error objective.
- Distinguish a teaching approximation from GPTQ.

Lecture notes

Let X contain calibration activation rows and W be a linear layer's weights. Reconstruction loss is $||X(W-\hat{W})||^2$_F. Its curvature depends on X^TX, so errors in frequently activated directions matter more than errors in rarely used directions. Correlations also matter: dropping off-diagonal terms changes the objective.

A diagonal weighting alone does not change independent nearest-grid rounding when every weight keeps the same fixed grid: multiplying a coordinate's squared error by a positive scalar preserves its minimizer. To make weighting consequential, compare candidate scales or allocate precision across coordinates. Full GPTQ additionally uses sequential error compensation and curvature structure; do not label a diagonal experiment as a full implementation.

Board example

For activation covariance diag(100,1), weight error [0.1,0] costs 1, while error [0,0.1] costs 0.01. Equal raw weight errors can therefore produce a hundredfold difference in the calibration objective.

Lab procedure

Compare ordinary rounding with a scale search minimizing activation-weighted reconstruction loss. Use a damped covariance estimate and a separate evaluation set. Show a case where calibration distribution shift reverses the ranking.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the objective, scale choices, and held-out error.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Identify which GPTQ steps are absent and why independent weighted rounding on a fixed grid is insufficient.

Further reading

- GPTQ paper

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 37 - Lecture 37: Sweep quantization bit widths and plot perplexity vs compression

Session: 90 minutes plus an implementation assignment. Preparation: Day 35, Day 36.

Learning objectives

- Compute token-weighted perplexity.
- Compare measured storage across bit widths.

Lecture notes

Perplexity is exp of mean negative log-likelihood over evaluated target tokens. Keep tokenizer, held-out corpus, context windows, stride, and padding policy fixed. Quantize each candidate from the same original checkpoint rather than repeatedly quantizing already rounded weights. Exclude calibration examples from evaluation.

Compression should use actual artifact bytes or a clearly labeled theoretical estimate that includes scale overhead. A quality curve is not a serving-speed curve. Weight-only quantization may use floating-point activations and different kernels at different precisions. Report failures such as unsupported bit widths rather than interpolating invented results.

Board example

If two batches have total negative log-likelihood 20 over 10 tokens and 90 over 30 tokens, the corpus mean is $110/40=2.75$ and perplexity is $\exp(2.75)\approx 15.64$. Averaging the two batch perplexities gives the wrong token weighting.

Lab procedure

Evaluate one small model at its reference precision and two supported lower precisions on the same corpus. Save total loss and valid-token count per batch, artifact bytes, and a few task-specific examples.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit perplexity-versus-bytes and task-quality tables.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why perplexity comparisons across different tokenizers require special care.

Further reading

- Perplexity evaluation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 38 - Lecture 38: Simulate draft-target speculative decoding with acceptance sampling

Session: 90 minutes plus an implementation assignment. Preparation: Day 15, Day 31.

Learning objectives

- Implement acceptance and residual sampling.
- Verify the target distribution empirically.

Lecture notes

For a draft distribution q and target p, sample x from q and accept it with probability min(1,p(x)/q(x)). On rejection, sample from the normalized positive residual max(p-q,0). In a sequence, stop verification at the first rejection; later draft tokens conditioned on the rejected history cannot be committed. Handle all-accepted proposals with the algorithm's next-token step.

The correction is essential. A rule that accepts only matching argmax tokens describes a different, greedy procedure. For a toy categorical simulation, estimate output frequencies across many draws and compare them with p. Distribution agreement is statistical, so report sample count and uncertainty rather than expecting identical sampled sequences.

Board example

Let p=[0.8,0.2] and q=[0.5,0.5]. Token A is always accepted; B is accepted with probability 0.4. Accepted mass is [0.5,0.2], and the rejected mass 0.3 goes entirely to A through the residual. Output probabilities are exactly [0.8,0.2].

Lab procedure

Implement the two-token case using Python's random module, then add three-token distributions. Test p=q, disjoint support, and near-zero probabilities. Compare empirical frequencies and rejected mass with hand calculations.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit distribution checks and a cost model with draft and verification overhead.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why acceptance rate alone cannot establish speedup.

Further reading

- Fast Inference from Transformers via Speculative Decoding

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 39 - Lecture 39: Build a simple KV cache manager (block allocator, eviction policy)

Session: 90 minutes plus an implementation assignment. Preparation: Day 05, Day 09.

Learning objectives

- Maintain allocation invariants.
- Handle shared blocks and exhaustion.

Lecture notes

A paged cache manager maps each request's logical blocks to physical block IDs. A free list tracks unused blocks, and reference counts track blocks shared by multiple requests. Allocation and release must preserve the invariant that a block is either free or referenced, never both. Partial final blocks need special treatment when requests append tokens.

Eviction is safe only for cache entries whose use is governed by the manager's lifecycle; active execution cannot lose its KV data silently. Under exhaustion, admission can wait, reject, or explicitly preempt and later recompute. Define this policy before implementing allocation. Reusing a freed ID must not leave a stale request mapping pointing at new content.

Board example

With four physical blocks, request A owns [0,1] and B owns [2]. The free set is {3}. If C shares A's block 0, its reference count becomes two. Releasing A frees block 1 but not block 0, which C still needs.

Lab procedure

Implement allocate, retain, release, and append operations using a small in-memory simulator. Drive random request lifecycles and verify no duplicate free IDs, no negative references, and no use of released blocks.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit lifecycle traces for sharing, exhaustion, cancellation, and reuse.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how copy-on-write protects a partially filled shared block.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 40 - Lecture 40: Implement prefix caching with hash-based deduplication

Session: 90 minutes plus an implementation assignment. Preparation: Day 16, Day 39.

Learning objectives

- Construct context-sensitive cache keys.
- Define collision and invalidation behavior.

Lecture notes

A prefix block's identity depends on both its token contents and the preceding prefix. Hashing only the current block wrongly equates identical local tokens that follow different histories. Include model revision and other execution-affecting state in the namespace. A lookup must also honor any isolation policy between callers.

Use a chained hash or explicit prefix key and keep enough metadata to validate identity under the chosen collision model. Deduplication affects ownership: shared physical blocks need reference counting, and appending to a partial shared block may require copying. Invalidation must make old-version entries unavailable without corrupting active requests still using them.

Board example

Prefixes [1,2,7,8] and [3,4,7,8] have the same final two-token block [7,8], but different preceding context. Their final block keys must differ. Prefixes [1,2,7,8] and [1,2,7,8,9] may reuse the common complete blocks.

Lab procedure

Extend Day 39 with chained keys and namespaces. Test equal prefixes, equal suffixes with different histories, model-version changes, and an injected hash collision using a deliberately weak test hash.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit hit/miss traces and memory savings.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which entries survive invalidation and how active readers finish safely.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 41 - Lecture 41: Simulate tensor parallelism: split a matmul across N workers

Session: 90 minutes plus an implementation assignment. Preparation: Day 17, Day 32.

Learning objectives

- Derive row and column partitioning.
- Identify required collectives.

Lecture notes

For Y=XW, splitting W by output columns produces independent output slices that can be concatenated. Splitting the contracted dimension instead requires matching slices of X and summing partial products. These are different communication patterns. A distributed model alternates compatible layouts to avoid unnecessary gathering between every operation.

A CPU or single-process simulation verifies algebra but does not measure network scaling. Real costs include launch synchronization, collective latency, transfer volume, and imbalance. Keep a reference matrix multiplication and compare outputs within tolerance because different reduction orders may change floating-point rounding.

Board example

For X=[2,3] and W=[[1,4],[5,6]], Y=[17,26]. Column shards return [17] and [26]. Contracted-dimension shards produce [2,8] and [15,18], whose sum is [17,26]. Concatenating the latter would be incorrect.

Lab procedure

Implement both decompositions for random matrices, including dimensions not divisible by the worker count. Record shard shapes and the bytes needed for each gather or reduction. Add a configurable communication delay model.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit equality checks and predicted speedup versus workers.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why smaller local matrix multiplications can reduce compute efficiency.

Further reading

- PyTorch distributed execution

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 42 - Lecture 42: Benchmark ops:byte ratio in practice across matrix sizes

Session: 90 minutes plus an implementation assignment. Preparation: Day 06, Day 41.

Learning objectives

- Estimate matrix traffic.
- Explain deviations from roofline predictions.

Lecture notes

For a dense M×K by K×N product, count about 2MKN floating-point operations. A simple single-read traffic model is b(MK+KN+MN) bytes, where b is bytes per element and the output is written once. This is a lower-bound model; cache behavior, repeated reads, and implementation details change actual traffic.

Sweep shapes independently rather than treating matrix size as one scalar. Skinny decode-like products can have low reuse despite large weights. Align dtype and arithmetic conventions with the compute ceiling: counting sparse or mixed-precision peak rates against a different operation produces misleading efficiency claims.

Board example

For M=1, K=N=1,024 and b=2, FLOPs are 2,097,152 and ideal traffic is 2,101,248 bytes, about 1 FLOP/byte. Increasing M allows the same weight matrix to serve more rows, raising reuse if the implementation can exploit it.

Lab procedure

Benchmark a grid of M, K, and N values with warmup and synchronized timing. Plot achieved FLOP/s against estimated intensity and annotate shapes that perform poorly relative to neighboring points.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the operation/byte derivation and measured plot.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain whether the traffic estimate represents cache, device memory, or an analytical lower bound.

Further reading

- PyTorch benchmarking

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 43 - Lecture 43: CUDA profiling: profile a PyTorch model with `torch.profiler`

Session: 90 minutes plus an implementation assignment. Preparation: Day 07, Day 27.

Learning objectives

- Capture a representative trace.
- Map operators to a critical-path hypothesis.

Lecture notes

Profiling introduces overhead and can alter scheduling. Use a short representative window after warmup, include CPU and supported device activities, and label request stages. Operator self time excludes child operations; total time can overlap across threads or streams, so summing all durations does not necessarily equal elapsed wall time.

Begin with a question such as whether repeated small launches dominate decode. Examine shapes and call counts, then locate synchronization and gaps. A high-duration operator is a candidate, not proof of the system bottleneck. Confirm the hypothesis by changing one cause and measuring the unprofiled workload again.

Board example

Suppose a 40 ms request includes 10 ms of CPU preparation overlapping 8 ms of GPU work, followed by 30 ms of serialized GPU work. Summed CPU-plus-GPU duration is 48 ms, while the critical path is 40 ms. Overlap explains the discrepancy.

Lab procedure

Capture a short trace of the Day 31 decode loop with labeled prefill and decode regions. Compare uncached and cached operation counts. If GPU activity is unavailable, perform the CPU trace and state that limitation.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a trace screenshot or exported trace plus a before/after measurement.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why profiling a cold first call would answer a different question.

Further reading

- PyTorch profiler recipe

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 44 - Lecture 44: Write a custom elementwise CUDA kernel via Triton

Session: 90 minutes plus an implementation assignment. Preparation: Day 07, Day 19, Day 42.

Learning objectives

- Map program IDs to array offsets.
- Mask tail loads and stores.

Lecture notes

An elementwise kernel partitions a flat array into blocks. Program ID selects a block, local offsets select elements within it, and a mask excludes offsets at or beyond the array length. Guard both loads and stores; a correct result for divisible sizes does not prove tail safety. Establish input dtype, device, contiguity, and aliasing assumptions.

Use a simple affine or vector-add operation first. Compare against a library reference, then tune block size across small and large arrays. JIT compilation belongs to startup cost and should be excluded from warm measurements. A custom kernel is useful only if its correctness and measured behavior justify maintaining it.

Board example

For N=1,000 and block size 256, launch four programs. The last covers offsets 768-1,023, of which only 232 are valid. Its mask must reject the remaining 24 positions. For N=0, define an empty-result path without an invalid launch.

Lab procedure

Implement a vector-add kernel following the official tutorial, then adapt it to y=ax+b. Test N=0,1,255,256,257, and 1,000, plus multiple dtypes. Report compilation and warm execution separately.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit tail tests and a bandwidth plot.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why increasing block size indefinitely may worsen performance.

Further reading

- Triton vector-add tutorial

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 45 - Lecture 45: Build a PyTorch custom op with CUDA backend

Session: 90 minutes plus an implementation assignment. Preparation: Day 08, Day 44.

Learning objectives

- Specify an operator contract.
- Integrate CUDA execution with dispatch.

Lecture notes

A custom operator needs a public contract before a kernel: supported ranks, shapes, dtypes, devices, strides, mutation, and output layout. Its registration must reflect those semantics. Reject unsupported inputs explicitly instead of interpreting arbitrary tensor memory as contiguous. A CPU reference is useful for correctness even when production use is GPU-only.

Device guards and the framework's current stream matter for composability. Register appropriate metadata behavior when using compilation, and state whether autograd is supported. An inference-only operator should not silently produce incorrect gradients. Packaging must account for compiler and framework compatibility; record the tested environment rather than claiming universal binary portability.

Board example

An operator that computes z=x+y for two length-N tensors must reject shapes [N] and [N,1] if broadcasting is not in its contract. Accepting both and returning a flat N-element array would disagree with standard broadcast semantics.

Lab procedure

Build a small operator from the official example, document its supported subset, and compare against a reference for valid inputs. Test noncontiguous views, mixed devices, mismatched shapes, and compilation metadata if supported.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the contract, correctness cases, and installation instructions for the tested environment.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain one failure that numerical happy-path tests alone would miss.

Further reading

- PyTorch custom C++ and CUDA operators

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 46 - Lecture 46: Deploy vLLM on spark-01, benchmark TTFT and throughput

Session: 90 minutes plus an implementation assignment. Preparation: Day 09, Day 27, Day 29.

Learning objectives

- Establish a reproducible serving baseline.
- Measure latency under controlled load.

Lecture notes

A serving benchmark includes frontend parsing, scheduling, model execution, and transport. Record the exact runtime and model revision, available memory, numerical format, and request distribution. Check the selected model and runtime against the actual Spark software stack before installation; a CPU-only analytical exercise cannot establish GPU throughput.

Begin with one request to establish correctness and a low-load baseline. Then increase offered concurrency while keeping prompt and output distributions fixed. Report failures and queue growth alongside successful throughput. Time-to-first-token and inter-token gaps must be measured from the stream, accounting for possible event buffering.

Board example

If 100 successful requests each produce 50 output tokens over a 10-second measurement interval, aggregate output throughput is 500 tokens/s. This does not imply that each request receives a token every 2 ms; concurrency and scheduling determine individual gaps.

Lab procedure

On an available supported host, follow the pinned runtime's official serving guide, load a small model, and replay a fixed fixture at concurrency 1, 2, 4, and 8. Save startup logs, configuration, and raw per-request events.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit correctness samples and latency/throughput curves. If execution is unavailable, submit the launch manifest and analytical capacity worksheet without reporting a measured benchmark.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Which assumption in the vllm serving experiment example most strongly affects its conclusion?

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 47 - Lecture 47: Deploy SGLang, benchmark structured output latency

Session: 90 minutes plus an implementation assignment. Preparation: Day 10, Day 29, Day 46.

Learning objectives

- Evaluate schema and semantic correctness.
- Measure grammar-related overhead.

Lecture notes

Structured decoding restricts the output space, but schema validity and task success remain different objectives. Specify the exact schema, validation policy, and semantic rubric before benchmarking. Compare against an unconstrained baseline on the same prompts and model; a different output length or retry policy can otherwise explain the observed latency difference.

Schema processing may introduce setup work, and repeated schemas may benefit from reuse. Separate first-use and warm-schema measurements. Include difficult cases such as empty arrays, optional fields, escapes, and truncated generations. A generation stopped by a token budget may remain incomplete even when the decoding process enforced valid prefixes.

Board example

Out of 100 outputs, suppose 98 pass JSON parsing, 95 pass the schema, and 80 contain the correct answer. Report 98%, 95%, and 80% separately. Calling this 95% task accuracy would conflate structure with meaning.

Lab procedure

Serve a supported model with SGLang using the documented structured-output interface. Evaluate a fixed extraction task with constrained and unconstrained generation, measuring cold/warm schema latency, completion rate, and semantic correctness.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the schema, fixtures, three correctness metrics, and latency distributions.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how you score truncated or missing responses.

Further reading

- SGLang structured outputs

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 48 - Lecture 48: TensorRT-LLM: compile a model and compare with eager PyTorch

Session: 90 minutes plus an implementation assignment. Preparation: Day 08, Day 11, Day 27.

Learning objectives

- Separate engine preparation from steady-state inference.
- Test the supported shape envelope.

Lecture notes

Compilation and runtime optimization trade preparation work for reusable execution plans. TensorRT-LLM has version-dependent execution paths; identify the documented path for your installed version and hardware instead of assuming every model follows one engine-build command. Use the same weights, tokenizer, sampling policy, and quality checks as the reference.

Record conversion, build, load, and warmup separately. Test shapes near the configured limits and define the response to an unsupported request. Numerical changes should be evaluated on logits or task quality as appropriate, not hidden by choosing only easy prompts. Compare total job time as well as warm latency.

Board example

An optimization requiring 180 seconds of setup and saving 6 ms per request breaks even after 30,000 requests, ignoring other differences. For a 1,000-request job, the warm saving is only 6 seconds, far below the setup cost.

Lab procedure

Choose a small supported model and build or configure the documented optimized path. Measure preparation time, warm latency, memory, and quality against eager execution at two prompt lengths. Record unsupported paths explicitly.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit an amortization curve and compatibility table.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which result would justify this approach for a long-running service but not a short batch job.

Further reading

- TensorRT-LLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 49 - Lecture 49: NVIDIA Dynamo: run a disaggregated prefill experiment

Session: 90 minutes plus an implementation assignment. Preparation: Day 12, Day 18, Day 46.

Learning objectives

- Measure KV-transfer overhead.
- Compare equal-resource serving layouts.

Lecture notes

Separate prefill and decode workers only after establishing a colocated reference. Keep total device count and workload fixed when comparing layouts; adding hardware changes the question. Instrument admission, prefill execution, transfer readiness, decode admission, and first-token delivery so the transfer is visible rather than buried in one aggregate timer.

Cache handoff must identify the request, model state, and correct token positions. Test a worker failure during transfer and define whether the request is retried, recomputed, or failed. Use the installed Dynamo backend's documented configuration; support varies with backend and hardware. A network estimate is not a measured transport benchmark.

Board example

Suppose splitting saves 45 ms of contention but adds 18 ms transfer and 12 ms coordination. The predicted improvement is 15 ms. If transfer doubles under load to 36 ms, the same layout becomes 3 ms slower than the reference.

Lab procedure

Run or design two equal-resource layouts using a fixed mix of short and long prompts. Sweep network bandwidth in a simulator if physical deployment is unavailable. Produce stage timelines and inspect one interrupted handoff.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the crossover plot, resource accounting, and failure outcome.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which workload change makes separation less attractive.

Further reading

- Dynamo documentation and release selection

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 50 - Lecture 50: Simulate continuous batching: queue arrivals, dynamic batch formation

Session: 90 minutes plus an implementation assignment. Preparation: Day 09, Day 23, Day 24.

Learning objectives

- Model arrivals and token-step scheduling.
- Evaluate admission and completion behavior.

Lecture notes

A discrete-event simulator can represent request arrival, prefill, decode steps, and completion. A static batch waits for its members according to a chosen policy; continuous batching admits new work as capacity becomes available. Define capacity in both active requests and token or memory budget, because equal request counts may represent very different work.

The scheduler needs explicit ordering for simultaneous events and a deterministic random seed. Keep service-time assumptions visible: fixed time per token is a simplification, while real step duration depends on batch and context. Include cancellation and exhausted token budgets so the simulation does not accumulate immortal requests.

Board example

With two active slots, A needs two decode steps and B needs five. If C arrives after step one, a continuous scheduler can admit C after A completes at step two. A static scheduler that waits for the whole batch delays C until B finishes at step five.

Lab procedure

Implement a small event loop and compare static admission with per-step admission on the same arrival trace. Record first service, first token, completion, and active batch size. Add one burst and one long-output outlier.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit scheduler timelines, queue lengths, and p95 latency by request class.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which conclusions depend on the assumed batch service-time function.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 51 - Lecture 51: Visualize PagedAttention block layout and fragmentation

Session: 90 minutes plus an implementation assignment. Preparation: Day 09, Day 39, Day 50.

Learning objectives

- Distinguish logical and physical blocks.
- Quantify internal fragmentation.

Lecture notes

A request's tokens occupy logical positions, while a page table maps those positions to physical blocks. Logical adjacency does not require physical adjacency. This permits reuse of freed blocks and sharing of compatible prefixes. Tail slots within a partially occupied block are internal fragmentation; free whole blocks are available capacity, not internal waste.

Draw the page table separately from physical storage. Visuals that place every request contiguously can obscure the reason for paging. Reference counts and copy-on-write must be visible when a shared partial block changes. Increasing block size reduces mapping overhead but may increase tail waste, so neither extreme is universally best.

Board example

For block size 8 and request lengths 9, 16, and 3, allocation is 2+2+1=5 blocks or 40 token slots. Useful occupancy is 28 tokens and internal waste is 12 slots, a 30% allocation waste fraction.

Lab procedure

Extend the Day 39 allocator with a text grid or plotted physical-block diagram. Animate or snapshot allocation, sharing, completion, and reallocation. Sweep block sizes 4, 8, and 16 on the same length distribution.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit diagrams and useful/allocated/free capacity counts.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why a large free region is not itself evidence of fragmentation.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 52 - Lecture 52: Benchmark TTFT vs throughput tradeoff across batch sizes

Session: 90 minutes plus an implementation assignment. Preparation: Day 27, Day 46, Day 50.

Learning objectives

- Construct a Pareto frontier.
- Choose a configuration under an explicit latency target.

Lecture notes

Batching can improve aggregate throughput while increasing waiting time or per-step latency. Sweep concurrency, scheduling limits, and batch policy one at a time. Distinguish the client's request batch from the runtime's internal token batch. Keep prompt and output distributions fixed and include failed or unfinished work in the experiment report.

A point is dominated if another configuration improves one objective without worsening the other. The useful frontier depends on service targets and quality constraints. A throughput maximum that violates an interactive latency target may be unsuitable, even if its utilization is impressive. Report repeated runs because frontier membership may be uncertain near measurement noise.

Board example

Configurations A, B, and C deliver (100 tokens/s, 50 ms p95 TTFT), (150,70), and (120,90). C is dominated by B. Under a 60 ms target, only A is eligible; under 80 ms, B becomes the higher-throughput eligible choice.

Lab procedure

Use the Day 46 fixture or Day 50 simulator to sweep at least four load levels. Plot aggregate useful output throughput against p95 TTFT, annotate errors, and mark the latency target before selecting a point.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the frontier and chosen configuration with uncertainty.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why comparing different output-length mixes would invalidate the selection.

Further reading

- PyTorch benchmarking

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 53 - Lecture 53: Write a production Dockerfile for a vLLM inference server

Session: 90 minutes plus an implementation assignment. Preparation: Day 22, Day 46.

Learning objectives

- Define image and model boundaries.
- Validate readiness and shutdown.

Lecture notes

A serving image should identify the runtime, dependency versions, entrypoint, and expected model location. Pin a tested base image by digest for reproducibility and record the target architecture. A model stored outside the image speeds image iteration but makes artifact availability and revision checking part of startup. Never equate a process listening on a port with a loaded, usable model.

Readiness should verify that the service can accept its intended workload; liveness should detect an unrecoverable process failure without repeatedly killing a slow load. Signal handling must reach the serving process, stop admission, and allow active work to drain. Keep secrets out of image layers and build arguments that become image history.

Board example

An image downloads in 20 seconds, model weights load in 45, and warmup takes 5. Sequential readiness is reached at 70 seconds, not at the first listening socket. If cached model artifacts reduce the measured weight-loading stage to 10 seconds, readiness moves to 35 seconds. Cached artifacts still need deserialization and device placement; measure these paths separately.

Lab procedure

Write a Dockerfile and startup contract for the Day 46 server using its supported image. Build locally if available, inspect the resulting configuration, and test readiness during loading plus termination during a long request.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the image digest, model revision, startup timeline, and shutdown outcome.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how the same image can still behave differently when model or runtime configuration changes.

Further reading

- Docker build practices

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 54 - Lecture 54: NVIDIA NIM deployment and custom serving-container contracts

Session: 90 minutes plus an implementation assignment. Preparation: Day 22, Day 53.

Learning objectives

- Distinguish a vendor NIM from a custom service.
- Version deployment artifacts.

Lecture notes

NVIDIA NIM packages a supported inference stack with model- and platform-specific deployment requirements. Matching a familiar HTTP endpoint does not turn an arbitrary image into a NIM. Choose either a documented NIM deployment on supported hardware or a custom container with an explicitly documented API contract. Describe that choice accurately in the artifact name and report.

For a NIM path, record the selected image, supported model profile, host requirements, and model-cache location from the official guide. For a custom path, define endpoint compatibility and test it rather than asserting vendor certification. Registry publication is a separate lifecycle step: a tag is mutable, while a digest identifies the pushed image content.

Board example

If tag course-server:latest points to digest A today and B tomorrow, repeating the tag does not reproduce the deployment. Recording digest A plus the model revision and configuration makes the intended artifact set unambiguous.

Lab procedure

Prepare a deployment manifest for a documented NIM or the Day 53 custom image. Validate health and one inference request, then document the registry push/pull procedure for a lab registry. Publication is an exercise, not an action performed by these notes.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the compatibility checklist, digest manifest, and smoke-test result.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

State exactly which contract was tested and whether an actual NVIDIA NIM was used.

Further reading

- NVIDIA NIM deployment playbook

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 55 - Lecture 55: Simulate an autoscaling policy: requests per second → replica count

Session: 90 minutes plus an implementation assignment. Preparation: Day 23, Day 50.

Learning objectives

- Model delayed capacity.
- Compare hysteresis and warm reserves.

Lecture notes

Start with a discrete-time queue model: backlog at the next step is max(0, backlog + arrivals - service capacity). New replicas contribute capacity only after startup completes. The controller observes a lagged signal and selects a desired replica count subject to minimum, maximum, and cooldown constraints. Scheduling a replica is not the same as making it ready.

Compare policies on identical arrival traces. A target utilization below one provides headroom but costs idle capacity. Scale-down must account for draining work and a minimum stable observation window. Prevent duplicate scale-up requests from repeatedly counting the same pending replica, a common error in simplified controllers.

Board example

At 30 arrivals/s and 20 service completions/s, backlog grows by 10 requests/s. A new 10-request/s replica arriving after 20 seconds finds 200 queued requests. Matching arrivals then stops growth but does not clear that backlog without additional spare capacity.

Lab procedure

Implement fixed-capacity, reactive, and warm-reserve policies. Include startup delay, a short burst, a sustained step, and a quiet period. Plot pending versus ready replicas, backlog, and approximate wait time.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit total replica-seconds and latency-target violations.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why a policy can stabilize the queue while leaving users waiting too long.

Further reading

- Kubernetes horizontal autoscaling

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 56 - Lecture 56: Measure cold start latency: model load times at different sizes

Session: 90 minutes plus an implementation assignment. Preparation: Day 22, Day 27, Day 53.

Learning objectives

- Decompose startup latency.
- Compare genuinely cold and cached conditions.

Lecture notes

Cold start consists of several possible stages: scheduling, image transfer, process initialization, model artifact transfer, deserialization, device placement, compilation, and warmup. Some overlap and some disappear on a cached host. Record each boundary rather than attributing the whole interval to model loading.

Define cold precisely. Restarting a process while the model remains in the filesystem cache is not an uncached download test. Repeat trials, record cache conditions, and avoid clearing shared caches merely to obtain a measurement. Use isolated test directories or fresh workers where available. A startup succeeds only when the documented readiness contract is met.

Board example

If download takes 40 seconds and initialization takes 10 after download, readiness cannot precede 50 seconds. If a later run uses cached weights and takes 12 seconds, the apparent 4.17× improvement describes cache state, not necessarily a faster loader.

Lab procedure

Measure a small and larger available model under process-cold and artifact-cached conditions. Collect stage timestamps and first successful inference. If the hardware cannot fit a model, report that constraint rather than extrapolating a load time.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit stacked timelines and a startup definition.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which optimizations reduce image, artifact, and device initialization costs respectively.

Further reading

- PyTorch benchmarking

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 57 - Lecture 57: Implement round-robin and least-connections load balancers

Session: 90 minutes plus an implementation assignment. Preparation: Day 24, Day 29.

Learning objectives

- Track active requests correctly.
- Compare round-robin and least-connections.

Lecture notes

Round-robin assigns arrivals cyclically across eligible backends. Least-connections selects a backend with the fewest active requests, requiring accurate increments at admission and decrements on completion, failure, or cancellation. Tie-breaking should be deterministic for tests. Remove unhealthy backends from eligibility without losing accounting for their in-flight requests.

Connection count may differ from request count under persistent or multiplexed transports. Even accurate active-request counts ignore remaining work: a single long generation can exceed several short jobs. Therefore evaluate both policies on heterogeneous lengths and treat load estimates as approximations. A retry must not leave a phantom active request on the failed backend.

Board example

If A has two jobs with one second remaining each and B has one ten-second job, least-active-request routing chooses B despite its larger outstanding work. Round-robin may choose either; neither policy uses the information needed to minimize estimated wait.

Lab procedure

Implement both policies against mock workers with configurable durations. Inject failures and cancellations and assert every active count returns to zero. Compare per-worker assignments and request latency for uniform and mixed jobs.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit policy traces and accounting invariants.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how backend removal interacts with requests already running there.

Further reading

- aiohttp client guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 58 - Lecture 58: Build a priority request queue with batch formation

Session: 90 minutes plus an implementation assignment. Preparation: Day 24, Day 50, Day 57.

Learning objectives

- Combine priorities with token budgets.
- Prevent starvation.

Lecture notes

A batch scheduler chooses eligible requests under capacity limits. A request-count limit alone may admit too many prompt tokens or too much KV state. Track an estimated token or memory budget and reserve room for ongoing decoding. Priority can reflect service class, but lower-priority work still needs an explicit progress policy.

Aging increases effective priority with wait time; weighted fair scheduling assigns shares across classes. Neither is interchangeable with FIFO. Stable ordering within a class makes behavior predictable. Large jobs that never fit the current residual budget need an explicit path, such as a dedicated slot or rejection with a documented limit.

Board example

With a 100-token admission budget, queued prompts of 70, 40, and 20 tokens cannot all enter. Selecting 70 and 20 uses 90; selecting 40 and 20 uses 60. A greedy fill rule changes fairness and may repeatedly postpone the 70-token request.

Lab procedure

Extend Day 50 with two priorities, stable arrival ordering, aging, and a token budget. Replay a continuous high-priority stream plus occasional large low-priority requests. Include one request larger than the total budget.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit wait-time distributions by class and a starvation trace before and after aging.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how overload differs from unfair scheduling.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 59 - Lecture 59: Multi-GPU tensor parallel benchmark across spark-01 and spark-02

Session: 90 minutes plus an implementation assignment. Preparation: Day 17, Day 21, Day 41.

Learning objectives

- Verify network topology.
- Separate model-fit gains from speedup.

Lecture notes

Two physical hosts communicate over their actual interconnect, not an assumed same-board GPU link. Inventory device architecture, network interface, transport support, runtime version, and collective backend before attempting distributed execution. Both workers need compatible model and tokenizer artifacts. Failure to form the distributed group is a setup result, not a zero-throughput benchmark.

Establish a single-device reference when the model fits. If it does not fit, compare model capacity separately from speed: the two-device run enables a previously impossible configuration but has no measured one-device latency baseline. Record communication and compute overlap, plus rank imbalance. Use fixed workloads and equivalent numerical formats.

Board example

If each decode step spends 6 ms on local compute and 4 ms on non-overlapped communication, step time is 10 ms. Reducing compute to 3 ms with more shards yields 7 ms only if communication stays constant; doubling communication instead gives 11 ms.

Lab procedure

Prepare and, where supported, execute a two-host run on spark-01 and spark-02. Save topology, startup diagnostics, and a small correctness fixture before load testing. Otherwise simulate communication costs using the Day 41 decomposition.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit speedup or capacity-enablement results with their distinct baselines.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why a faster network may matter more than another compute shard.

Further reading

- PyTorch distributed execution
- DGX Spark user guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 60 - Lecture 60: MIG capability checks and partition planning on supported GPUs

Session: 90 minutes plus an implementation assignment. Preparation: Day 21, Day 55.

Learning objectives

- Check explicit hardware support.
- Evaluate service placement under partition limits.

Lecture notes

MIG partitions supported GPUs into hardware instances with defined resource profiles. Support is product-specific; compute capability or the word Blackwell alone is insufficient. The original roadmap's instruction to configure MIG on Spark is replaced with capability discovery and supported-device practice. DGX Spark GB10 is not listed in the cited MIG supported-products table; use a simulator unless the actual product is explicitly supported.

Model each partition as a capacity and compute allocation rather than a fraction of the parent's advertised throughput. Profiles constrain feasible combinations. Partition reconfiguration affects running workloads and belongs in a controlled lab procedure. On unsupported hardware, process-level concurrency experiments are not equivalent to MIG isolation.

Board example

A service needs 10 GiB of weights, 5 GiB of cache, and 2 GiB of workspace: 17 GiB before extra headroom. It cannot fit a 10 GiB profile. Two 10 GiB instances do not transparently supply one 20 GiB address space.

Lab procedure

Record the exact device identity and compare it with the official support table. Simulate placing three services into candidate profiles. Only on a dedicated supported device, follow the vendor setup procedure and compare isolation under concurrent load.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit capability evidence, feasible placements, and simulated or measured latency.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why memory fit alone does not prove sufficient compute capacity.

Further reading

- NVIDIA MIG supported products

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 61 - Lecture 61: GPU cost model: $/token across instance types at different utilizations

Session: 90 minutes plus an implementation assignment. Preparation: Day 26, Day 52.

Learning objectives

- Define a consistent cost denominator.
- Analyze utilization without double counting.

Lecture notes

Cost per million useful tokens equals total cost over the observation window divided by delivered useful tokens, multiplied by one million. Include all replicas, idle capacity, and relevant infrastructure if claiming total service cost. Define input versus output accounting and how failed, retried, or discarded tokens are treated.

A prospective model can multiply peak throughput by an assumed effective utilization. An observed average throughput already includes the measured idle time, so multiplying it by utilization again double-counts the loss. Keep hypothetical prices labeled and date any live quotes. Compare cost only among configurations that meet the same quality and latency constraints.

Board example

A fleet costs $18 during one hour and delivers 3.6 million useful output tokens. Cost is $5 per million output tokens. If raw execution produced 4 million but 0.4 million were discarded, dividing by 4 million understates cost per useful output.

Lab procedure

Build a worksheet with hourly fleet cost, observed token count, quality pass rate, and latency eligibility. Sweep utilization only in the prospective model. Compare warm-reserve and scale-to-zero scenarios using explicitly hypothetical prices.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit sensitivity plots and an accounting boundary.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how retries can increase both billed compute and apparent raw token throughput while worsening useful cost.

Further reading

- Google SRE release engineering

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 62 - Lecture 62: Blue-green deployment: zero-downtime model version swap

Session: 90 minutes plus an implementation assignment. Preparation: Day 26, Day 53, Day 57.

Learning objectives

- Gate traffic on readiness.
- Preserve streaming requests during cutover.

Lecture notes

Blue-green deployment prepares a complete new serving pool before changing routing. Validate model identity, readiness, and representative inference results before sending ordinary traffic. A successful health response alone does not establish model quality. Record the old and new artifact sets so rollback restores a known configuration.

Routing changes affect new admissions; existing streams should finish on their original worker where the infrastructure permits. Drain old workers by stopping admission and tracking active requests until completion or an explicit deadline. A rollback can redirect future requests but cannot erase text already streamed to a client. Test that limitation rather than promising perfect invisibility.

Board example

Suppose the old pool has 12 active streams and accepts no new work after cutover. Ten finish within 20 seconds and two need 80 seconds. A 30-second drain deadline interrupts two requests unless the deployment keeps those workers alive or defines an explicit recovery behavior.

Lab procedure

Use mock pools or an available lab server to roll from model version A to B. Include a readiness failure, a bad quality canary, and a long stream. Record admission decisions, version identity, and active counts throughout.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the cutover and rollback timeline plus interruption counts.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain the temporary capacity cost and the meaning of zero downtime for streams.

Further reading

- Google SRE release engineering

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 63 - Lecture 63: Emit Prometheus metrics from an inference server

Session: 90 minutes plus an implementation assignment. Preparation: Day 28, Day 53.

Learning objectives

- Choose counters, gauges, and histograms.
- Keep metrics internally consistent.

Lecture notes

Instrument arrivals, completions, and errors as counters; current queue depth and in-flight requests as gauges; and durations as histograms. Decide whether completion counters include failures and keep names or labels consistent with that definition. Time units should be explicit. Observe latency at a lifecycle boundary that occurs on all relevant outcomes.

Bound label cardinality with a small set of model versions, outcomes, or service classes. User IDs, request IDs, and prompt text produce unbounded series and are unsuitable metric labels. On cancellation or exception, decrement gauges in a guaranteed cleanup path. In a multiprocess service, configure collection according to the client library's documented process model.

Board example

Over an interval, 100 requests arrive, 90 succeed, 5 fail, and 5 remain active. If the system began empty, arrivals minus terminal completions equals 5 in flight. A counter that counts only successes cannot be used alone to reconcile active work.

Lab procedure

Instrument a mock server and send a known sequence of successful, failed, and cancelled requests. Scrape its metrics and reconcile arrivals, terminal outcomes, and current gauges. Check that each request observes duration once.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the metric schema, sample exposition, and lifecycle reconciliation.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain what a restart does to counters and how rate calculations should handle it.

Further reading

- Prometheus instrumentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 64 - Lecture 64: Build a Grafana dashboard: TTFT, TBT, queue depth, GPU utilization

Session: 90 minutes plus an implementation assignment. Preparation: Day 28, Day 63.

Learning objectives

- Build panels around diagnostic questions.
- Aggregate distributions correctly.

Lecture notes

A useful dashboard starts with user-visible service health and then shows likely causes. Pair request rate, error rate, and latency with queue depth, active work, cache behavior, and device telemetry. Use consistent time windows and units so correlated changes remain interpretable. An empty panel should distinguish missing telemetry from a genuine zero.

Fleet percentiles require aggregation of compatible histogram buckets before quantile estimation. Do not average replica p95 values. A rate increase can explain a queue rise without any model slowdown, while flat arrivals with rising execution duration suggests a different cause. Annotate deployments and configuration changes to test these hypotheses.

Board example

Worker A handles 900 fast requests and worker B handles 100 slow ones. Weighting their p95 values by request counts still does not reconstruct the fleet p95; quantiles are nonlinear. The underlying bucket counts or observations are needed.

Lab procedure

Create a Grafana dashboard specification or local dashboard from Day 63 metrics. Include TTFT, inter-token gaps, completion latency, queue depth, error rate, and device utilization. Replay a burst, a slow worker, and a failed scrape.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit panel definitions and three annotated incident views.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which panels distinguish demand growth from reduced service capacity.

Further reading

- Grafana dashboard practices

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 65 - Lecture 65: Add distributed tracing (OpenTelemetry) to an inference request

Session: 90 minutes plus an implementation assignment. Preparation: Day 28, Day 63.

Learning objectives

- Propagate request context.
- Interpret overlapping spans.

Lecture notes

A trace links spans across client, router, scheduler, model worker, and response transport. Propagate the context through queued work and asynchronous tasks; otherwise the slow stage may appear as an unrelated root trace. Record stage names, timing, bounded attributes, and terminal status without copying prompt content into routine traces.

Parent-child duration is not additive when children overlap. Trace sampling can omit rare slow requests, so select a strategy consistent with the diagnostic goal and record its limits. Cross-host wall clocks can skew apparent order; use per-process monotonic durations and treat distributed timestamps carefully. A span around an async launch may not measure completed device work.

Board example

Two child operations lasting 40 and 60 ms start together, and the parent waits for both. Their combined critical-path contribution is 60 ms, not 100 ms. A further 10 ms of serialized response work yields a 70 ms parent duration.

Lab procedure

Trace a mock router-to-worker request with a queue stage and a streaming response. Inject a delayed worker and a cancelled client. Verify context propagation and that spans end on both normal and exceptional paths.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit two trace timelines and a critical-path explanation.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Identify which timing still requires device-level instrumentation.

Further reading

- OpenTelemetry traces

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 66 - Lecture 66: Load test with Locust: ramp traffic, find saturation point

Session: 90 minutes plus an implementation assignment. Preparation: Day 27, Day 52, Day 63.

Learning objectives

- Measure offered and completed load.
- Locate a sustainable operating point.

Lecture notes

Load testing should reveal where the service stops meeting its objectives. Define request mix, duration, ramp steps, maximum load, and stop conditions. A closed-loop user model can reduce arrival rate as latency grows, hiding the degree of overload. Report actual offered requests per second alongside configured user count and successful completion rate.

Streaming HTTP success does not guarantee successful generation. Parse terminal events and validate output so truncated streams count as failures. The load generator itself can saturate CPU, sockets, or network; inspect its resource use before attributing the plateau to the server. Include a drain period to account for queued requests after arrivals stop.

Board example

At offered load 100 requests/s and completion capacity 80/s, backlog grows by about 20/s. A 60-second steady overload creates roughly 1,200 queued requests under a fluid model. Reporting only successful completions conceals that accumulating debt.

Lab procedure

Create a Locust workload or equivalent fixture-aware generator with staged load. Track offered load, completions, failed streams, p95 TTFT, and queue slope. Stop when a predefined error or latency threshold is exceeded in the lab.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the saturation curve and sustainable load recommendation.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how you verified the generator was not the bottleneck.

Further reading

- Locust workload authoring

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 67 - Lecture 67: Profile with Nsight Systems: identify kernel launch overhead

Session: 90 minutes plus an implementation assignment. Preparation: Day 07, Day 43.

Learning objectives

- Identify launch gaps and synchronization.
- Validate a critical-path optimization.

Lecture notes

A system timeline relates host activity, runtime calls, device kernels, and transfers. Focus capture on a warmed representative region to avoid drowning the question in initialization. Add stage markers so a cluster of kernels can be tied back to prefill, decode, or data movement. Observe both the host and device before blaming kernel duration.

Launch overhead matters when useful kernels are short and separated by host-side gaps. Synchronization can serialize otherwise independent work, but apparent empty device time may also reflect dependencies or missing instrumentation. A system profiler identifies where to investigate; detailed kernel efficiency may require another tool. Measure the final improvement without capture overhead.

Board example

Suppose each step has 20 kernels of 5 microseconds plus 20 non-overlapped launch gaps of 5 microseconds. The simplified total is 200 microseconds, half in gaps. Removing half the kernels and gaps gives 100 microseconds only if the fused kernels retain the assumed total work behavior.

Lab procedure

Capture a short decode loop with Nsight Systems on supported hardware. Identify one launch-bound or synchronization-heavy region. Compare a batched or compiled variant and remeasure normal execution outside the profiler.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit an annotated timeline, hypothesis, and unprofiled comparison.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

State which intervals overlap and which are serialized.

Further reading

- Nsight Systems user guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 68 - Lecture 68: Build a streaming inference client using SSE

Session: 90 minutes plus an implementation assignment. Preparation: Day 29, Day 65.

Learning objectives

- Parse event framing across arbitrary reads.
- Handle completion and interruption.

Lecture notes

Server-sent events use a text framing protocol whose event boundaries are independent of TCP chunks. Keep a byte/text buffer, decode UTF-8 incrementally, collect fields until a blank line, and dispatch the resulting event. Multiple data fields form one event payload. Comments may act as keepalives and should not become generated text.

The application defines how event payloads represent tokens, errors, and completion. A completion marker such as a sentinel string is application-specific, not a universal SSE rule. Distinguish a clean application terminal event from a transport close before completion. Automatic reconnect cannot safely resume arbitrary generation unless the service defines an explicit resume contract.

Board example

A read may end with data: {“te and the next begin with xt”:“hi”} followed by a blank line. Neither read is a complete JSON payload alone. Conversely, one read may contain several complete events that must all be processed.

Lab procedure

Build a parser fixture that splits every byte boundary of a multibyte message, combines multiple events, includes comments, and closes mid-event. Measure first application-content arrival separately from the first network byte.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit parser tests and interruption behavior.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how repeated events or reconnection would affect displayed output.

Further reading

- Server-sent events

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 69 - Lecture 69: Async batch inference client using **`asyncio`** + **`aiohttp`**

Session: 90 minutes plus an implementation assignment. Preparation: Day 29, Day 68.

Learning objectives

- Reuse connections.
- Preserve per-request results under partial failure.

Lecture notes

Use a shared client session and an explicit concurrency limit to avoid opening a new connection pool for every request. Bound the number of queued tasks as well as active network operations for large datasets. A semaphore around network work does not by itself prevent allocating millions of pending coroutine objects.

Give each input a stable index and store its outcome independently of completion order. Timeouts, cancellations, and parsing failures should produce structured results rather than silently disappearing. Retry only according to a stated policy, with bounded attempts and backoff; an ambiguous timeout may already have consumed server work. Cleanup must close responses and release permits on every path.

Board example

With 1,000 inputs, concurrency 10, and 0.5-second mean service time, an ideal no-overhead lower bound is about 50 seconds. A worker-pool design can keep only ten active requests plus a bounded input queue instead of 1,000 live tasks.

Lab procedure

Implement a producer and fixed worker pool against a mock HTTP endpoint. Inject latency, HTTP errors, malformed payloads, and cancellation. Verify output order by input ID and that active requests never exceed the configured bound.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit result records and peak concurrency/memory observations.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how you prevent retries from duplicating final results.

Further reading

- aiohttp client guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 70 - Lecture 70: Multi-cloud routing: geo-aware latency-based request routing

Session: 90 minutes plus an implementation assignment. Preparation: Day 25, Day 57, Day 61.

Learning objectives

- Combine network and queue estimates.
- Preserve constraints during failover.

Lecture notes

The nearest region by geography is not necessarily the fastest available route. Estimate network delay, queue delay, execution time, and any required artifact or cache transfer. Apply eligibility constraints before scoring regions, including supported model revision and required data placement. A low predicted latency does not override an incompatible backend.

Use health signals with hysteresis so transient failures do not cause rapid oscillation. Stale queue estimates can send many routers to the same apparently idle destination; consider dampening and admission feedback. Failover capacity must be ready or its cold-start delay belongs in the recovery estimate. Keep cross-region transfer costs in the service cost model.

Board example

Region A has 20 ms network delay and 100 ms queue wait; B has 70 ms network delay and 10 ms wait. With equal execution time, B is 40 ms faster despite being farther away. If B is an incompatible model version, it is ineligible regardless of that score.

Lab procedure

Simulate two regions with time-varying queues and one outage. Compare nearest-region routing with predicted completion time and constrained failover. Include delayed health updates and cold standby startup.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit per-region assignments, p95 latency, and recovery time.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain a case where stale state reverses the policy ranking.

Further reading

- Google SRE release engineering

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 71 - Lecture 71: GPU memory profiling: find where your memory budget goes

Session: 90 minutes plus an implementation assignment. Preparation: Day 05, Day 34, Day 63.

Learning objectives

- Reconcile weights, cache, and workspace.
- Distinguish live allocation from reservation.

Lecture notes

A serving memory budget includes weights, KV state, activations, temporary workspaces, runtime allocations, and fragmentation or allocator reserves. These categories have different lifetimes and scaling rules. Measure at model load, after warmup, during peak prefill, and during steady decode; one idle snapshot cannot characterize the peak.

Framework allocated memory counts live managed tensors, while reserved memory includes allocator-held capacity. Device totals may also include allocations outside the framework. Releasing Python references and clearing allocator caches have different effects; neither proves that a genuine live-tensor leak is fixed. On shared-memory hardware, record the system accounting method as well.

Board example

A hypothetical 24 GiB device holds 10 GiB weights, 8 GiB KV, and 4 GiB peak workspace, leaving only 2 GiB for other overhead. A 3 GiB prefill spike would exceed the nominal budget even though the idle model appears to fit comfortably.

Lab procedure

Instrument staged allocations for a small model and sweep batch/context sizes. Compare analytical KV bytes with measured changes. Repeat request cycles and observe whether live allocations return to a stable baseline.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a memory waterfall and capacity boundary.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how reserved-but-unused memory differs from a persistent live allocation leak.

Further reading

- PyTorch CUDA memory management

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 72 - Lecture 72: Benchmark quantization levels on real throughput: FP16 vs INT8 vs INT4

Session: 90 minutes plus an implementation assignment. Preparation: Day 13, Day 37, Day 46.

Learning objectives

- Compare supported precisions fairly.
- Enforce quality and latency constraints.

Lecture notes

Precision changes affect weight traffic, activation processing, dequantization, kernel choice, and numerical behavior. A smaller checkpoint does not guarantee faster execution. Verify that each runtime path actually uses the intended optimized kernels; a floating-point fallback may preserve output while defeating the expected acceleration.

Hold model family, input/output distribution, and serving resources fixed. Evaluate a reference, INT8, and INT4 only where supported, recording any non-quantized layers and scale overhead. Keep a task-quality floor and latency objective so performance is not improved by accepting unusable results. Report both single-request latency and saturated useful throughput.

Board example

Suppose FP16 delivers 1,000 useful tokens/s and INT4 delivers 900 despite smaller weights. If INT4 adds dequantization overhead or selects a less efficient kernel for the tested shape, the result is plausible. Storage compression alone cannot override that measurement.

Lab procedure

Run a fixed fixture across supported precisions with warmup and repeated trials. Capture artifact size, peak memory, quality pass rate, p95 TTFT, and useful output tokens/s. Inspect the selected runtime path when a result is surprising.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a precision Pareto table and one causal explanation supported by profiling. Mark unsupported configurations rather than filling their cells with projections.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Which assumption in the real quantized throughput example most strongly affects its conclusion?

Further reading

- GPTQ: quantization and reconstruction objectives
- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 73 - Lecture 73: Measure speculative decoding acceptance rates by draft model size

Session: 90 minutes plus an implementation assignment. Preparation: Day 15, Day 38, Day 72.

Learning objectives

- Measure acceptance and committed progress.
- Include draft overhead and memory.

Lecture notes

A larger draft model may agree with the target more often but also cost more to execute and store. Measure acceptance by position and prompt class, then measure committed tokens per complete verification cycle. Rejections truncate accepted prefixes, so a single marginal token agreement rate does not fully determine useful progress.

Keep target, sampling policy, prompt distribution, and draft length controlled. The draft can compete for memory or bandwidth with the target; compare total serving resources and concurrency. Include the case where target-only generation is faster. An exact-sampling implementation needs the correction rules from Day 38, not only a match-rate heuristic.

Board example

Draft A costs 2 ms plus 10 ms verification and commits 2 tokens on average: 6 ms/token. Draft B costs 6 ms plus 10 ms verification and commits 3: about 5.33 ms/token. If B commits only 2.5, it becomes 6.4 ms/token and loses to A.

Lab procedure

Compare at least two available draft sizes or a labeled cost simulation. Record per-position acceptance, committed tokens/cycle, draft time, verification time, total memory, and target-only latency.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the size-versus-effective-latency curve.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why the draft with the highest acceptance may not be the best system choice.

Further reading

- Fast Inference from Transformers via Speculative Decoding

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 74 - Lecture 74: Measure KV cache hit rates across real traffic patterns

Session: 90 minutes plus an implementation assignment. Preparation: Day 16, Day 40, Day 63.

Learning objectives

- Separate request and token hit rates.
- Account for cold state and eviction.

Lecture notes

A request-level hit metric asks whether any prefix was reused; a token-level metric measures how much prefill work may have been avoided. Both are useful but answer different questions. Weighting by reused tokens better reflects prefix length, while actual saved time also depends on device and scheduling. Always define the denominator.

Replay workloads in timestamp order because cache capacity, arrival patterns, and eviction change outcomes. Separate cold-start, warming, and steady intervals. Include model-version changes and tenant namespace rules. A high hit rate can coexist with poor latency if cache-aware routing concentrates demand on overloaded workers.

Board example

Two requests have prompt lengths 10 and 1,000. The first reuses all 10 tokens; the second reuses none. Request hit rate is 50%, but reused-token fraction is $10/1{,}010 \approx 0.99\%$. Reporting only the first number exaggerates the likely saved work.

Lab procedure

Replay repeated-template, chat-history, and unique-prompt traces through Day 40's cache. Sweep capacity and eviction policy. Record request hits, reused tokens, evictions, and estimated or measured prefill time saved.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit cold/warm time series and hit-rate definitions.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why a version change should affect reuse even for identical token IDs.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 75 - Lecture 75: Tensor parallelism scaling: throughput and latency vs GPU count

Session: 90 minutes plus an implementation assignment. Preparation: Day 17, Day 59, Day 72.

Learning objectives

- Distinguish strong scaling from capacity scaling.
- Report communication overhead.

Lecture notes

Strong scaling fixes the workload and increases devices, measuring speedup and efficiency. Capacity scaling asks what larger models or workloads additional devices make possible. Keep these experiments separate. Tensor parallelism may improve memory fit while increasing latency for small batches because collectives and smaller local operations dominate.

Record topology, rank placement, dtype, sequence lengths, and batch policy. Compare one, two, and more devices only when the model fits each configuration. Where it does not, state the missing baseline. Device cost and power rise with device count, so the fastest point may not minimize cost under a latency target.

Board example

If one device takes 100 ms and four take 40 ms, speedup is 2.5× and parallel efficiency is 2.5/4=62.5%. Four times the hardware has not produced four times the speed. A 25 ms ideal reference ignores communication and serial work.

Lab procedure

Run supported device counts or simulate local compute plus collective costs from Day 59. Sweep batch size at each count and record p95 latency, throughput, memory, and communication time.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit scaling and efficiency plots with explicit baseline availability.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which operating point fits the model and satisfies the service target at lowest modeled cost.

Further reading

- PyTorch distributed execution

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 76 - Lecture 76: End-to-end latency breakdown: tokenization → TTFT → TBT → detokenization

Session: 90 minutes plus an implementation assignment. Preparation: Day 27, Day 65, Day 68.

Learning objectives

- Define disjoint stage intervals.
- Reconcile client and server clocks.

Lecture notes

Client-observed latency includes transport, queueing, tokenization, prefill, token selection, decode, serialization, and delivery. Some stages overlap, so a decomposition must identify disjoint intervals or explicitly model a critical path. Time to first token already contains earlier stages and must not be added to them again as an independent component.

Use a consistent client monotonic clock for send, first content, and terminal event. Server spans explain internal time but may use different clocks. Separate time to first byte from first generated content, and note that a stream event can contain multiple tokens. Average token time inferred from totals is not a full distribution of inter-token gaps.

Board example

For a request with TTFT 120 ms and four further token gaps [20,30,20,30] ms, terminal content arrives at 220 ms, excluding any later completion metadata. If tokenization took 10 ms inside TTFT, adding it again would incorrectly report 230 ms.

Lab procedure

Instrument a mock or lab request with client events and server stage spans. Reconcile the timeline, mark overlap, and explain any unaccounted interval. Repeat with network buffering and one queue delay.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a latency waterfall and clock-boundary definitions.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why per-token averages can hide a long pause in the middle of a response.

Further reading

- OpenTelemetry traces

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 77 - Lecture 77: Build a reusable inference benchmark harness

Session: 90 minutes plus an implementation assignment. Preparation: Day 27, Day 66, Day 76.

Learning objectives

- Define a stable experiment schema.
- Preserve raw results and failures.

Lecture notes

A benchmark harness separates workload specification, transport adapter, execution scheduling, event recording, and analysis. Keep these boundaries small and concrete rather than building a general orchestration framework. Store enough metadata to reproduce the experiment: model revision, runtime, hardware, configuration, input IDs, timing conventions, and source revision.

Raw per-request events should remain immutable inputs to summaries. Record requested and actual token counts, terminal outcome, and timing fields with units. The analyzer should reject incompatible comparisons or missing required metadata. Timeouts and malformed streams are results, not reasons to silently shrink the denominator. Seed synthetic workloads and save the generated fixture.

Board example

If run A uses 100 requests with 100 output tokens each and run B uses 100 requests with 20 tokens each, comparing requests/s alone favors a different task. A harness can flag the mismatch before drawing a speed conclusion.

Lab procedure

Combine the Day 69 client, Day 68 parser, and Day 27 analysis into a small command-line experiment runner. Add a mock transport for deterministic failure and timing fixtures, then run one real supported endpoint if available.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a sample manifest, raw result file, summary, and reproduction command.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Demonstrate that partial failures survive into the final report.

Further reading

- PyTorch benchmarking

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 78 - Lecture 78: Vision Language Model (VLM) inference: image preprocessing and batching

Session: 90 minutes plus an implementation assignment. Preparation: Day 04, Day 27, Day 52.

Learning objectives

- Account for image preprocessing and visual tokens.
- Batch by actual multimodal cost.

Lecture notes

A vision-language request often includes image decoding, resizing or tiling, a vision encoder, projection into language-model inputs, and text generation. The exact sequence depends on the architecture. Use the model's own processor and record its configuration; arbitrary resizing can alter both token count and answer quality. Image count alone is a weak workload descriptor.

Batching may require padding visual and textual sequences, which wastes work when resolutions or tile counts differ. Bucket by compatible processing shapes and total token budget. Measure preprocessing separately so a CPU image bottleneck is not misdiagnosed as slow language decoding. Include corrupt images and unsupported dimensions in the input contract.

Board example

In a hypothetical patch encoder with 16×16 patches and no extra tokens, a 224×224 image gives $14^2=196$ patches. A 448×448 image gives $28^2=784$, four times as many. Actual models may add tokens, tile images, or compress features, so verify the processor output.

Lab procedure

Use a supported small VLM on the same image at two resolutions. Record processor output shapes, preprocessing time, encoder time where observable, TTFT, and answer quality. Compare mixed-resolution with bucketed batches.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the shape trace and cost/quality comparison.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why lowering resolution may improve speed while losing small-text or fine-detail accuracy.

Further reading

- Image-text-to-text inference

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 79 - Lecture 79: Embedding model inference: batching and throughput optimization

Session: 90 minutes plus an implementation assignment. Preparation: Day 03, Day 27, Day 52.

Learning objectives

- Choose pooling and normalization deliberately.
- Optimize token-aware batching.

Lecture notes

Embedding models convert token sequences into fixed-size vectors using a model-specific pooling and normalization policy. Padding must be excluded where the pooling rule requires it. CLS pooling, mean pooling, and normalized embeddings are not interchangeable choices; use the checkpoint's documented inference contract before experimenting.

Batch by total tokens or length buckets to reduce padding overhead. Report vectors per second and input tokens per second because request lengths vary. A throughput improvement is useful only if retrieval or similarity quality remains adequate. Tokenization, host-device transfer, and vector serialization may dominate smaller models, so include them in end-to-end measurements.

Board example

For sequence lengths 8, 8, and 64 in one padded batch, 192 token positions are processed for 80 real tokens: only 41.7% are useful under this simplified count. Grouping the short pair separately can reduce padding without changing their contents.

Lab procedure

Embed a fixed corpus using sequential, fixed-count batched, and length-bucketed execution. Verify output dimensions, normalization, and similarity rankings against a reference. Measure end-to-end throughput and peak memory.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit padding efficiency and retrieval-quality checks.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why comparing vectors/s across different length distributions can mislead.

Further reading

- Sentence Transformers inference

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 80 - Lecture 80: ASR (Whisper): single-chunk and long-file latency optimization

Session: 90 minutes plus an implementation assignment. Preparation: Day 27, Day 29.

Learning objectives

- Measure real-time factor and transcription error.
- Handle long-audio boundaries.

Lecture notes

ASR inference includes audio decoding, channel handling, resampling, feature extraction, and model execution. Preserve the expected sample rate and preprocessing contract. Duration is a natural workload unit, but speech density, language, silence, and noise also affect decoding behavior. Record these properties for evaluation fixtures.

Long recordings may require windows or chunks, with overlap or context to preserve words near boundaries. Independent chunk transcription can duplicate or omit boundary text. Word error rate counts substitutions, deletions, and insertions relative to the reference; punctuation and normalization policies affect the score. Report real-time factor together with first-result delay for streaming-like workflows.

Board example

Processing 60 seconds of audio in 12 seconds gives real-time factor 0.2. If the reference has 100 words and the transcript has 5 substitutions, 3 deletions, and 2 insertions, WER=(5+3+2)/100=10%. Faster execution alone says nothing about that error rate.

Lab procedure

Use a small supported Whisper checkpoint and short licensed or self-recorded clips. Compare whole-clip and chunked transcription, keeping preprocessing fixed. Inspect words crossing chunk boundaries and include a silence segment.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit duration, runtime, WER policy, and boundary examples.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why a low average real-time factor does not guarantee low interactive delay.

Further reading

- Whisper reference implementation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 81 - Lecture 81: TTS: streaming real-time text-to-speech

Session: 90 minutes plus an implementation assignment. Preparation: Day 29, Day 68, Day 80.

Learning objectives

- Separate first-audio latency from synthesis speed.
- Manage chunk and playback buffers.

Lecture notes

Text-to-speech may include text normalization, acoustic generation, and waveform synthesis. A streaming service emits playable audio chunks before the whole utterance is complete. Measure time to first audio, total synthesis time, and playback underruns separately. Audio duration provides the denominator for real-time factor, while chunk scheduling determines interactivity.

Small text chunks can reduce initial delay but deprive the model of context needed for pronunciation and prosody. Large chunks improve context at the cost of waiting. Preserve sample rate, channel count, and codec framing across chunks. A playback buffer absorbs jitter but adds latency; define what happens when a client cancels mid-utterance.

Board example

If synthesizing 10 seconds of audio takes 2 seconds, real-time factor is 0.2. Yet if the first playable chunk arrives after 1.5 seconds, the interaction still has a noticeable initial wait. A 200 ms playback buffer can absorb smaller production jitter but adds 200 ms to first playback.

Lab procedure

Use an available streaming TTS engine or a timed audio-chunk simulator. Sweep chunk sizes and buffer depth, then measure first playback, underruns, and total duration. For a real model, listen for boundary artifacts using the same text.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit latency and continuity results.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why faster-than-real-time average generation can still produce audible stalls.

Further reading

- Text-to-speech task guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 82 - Lecture 82: Image generation: diffusion model inference and kernel optimization

Session: 90 minutes plus an implementation assignment. Preparation: Day 07, Day 27, Day 72.

Learning objectives

- Decompose iterative generation cost.
- Evaluate quality when reducing work.

Lecture notes

Diffusion generation repeatedly updates a noisy state using a learned denoising model and a chosen sampling schedule. End-to-end work can also include text encoding, image decoding, safety or quality processing, and serialization. Keep the model, scheduler, seed policy, resolution, and guidance settings explicit when comparing configurations.

Reducing denoising steps changes the inference algorithm and may change quality; it is not equivalent to optimizing the same computation. Compilation, precision changes, and memory-efficient attention can improve execution, but effects depend on shapes and support. Image batches can increase throughput while increasing memory and single-image latency. Use multiple prompts and seeds rather than selecting one attractive output.

Board example

With 30 denoising steps at 20 ms each plus 100 ms fixed overhead, total time is 700 ms. Reducing step time to 15 ms gives 550 ms. Reducing the count to 15 gives 400 ms under the original timing, but requires a separate quality evaluation.

Lab procedure

Benchmark a supported small diffusion pipeline across two step counts and two execution settings. Hold prompt/seed fixtures fixed and record image quality judgments, total time, denoiser time, and peak memory.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a quality/latency grid and identify which changes alter the algorithm.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain the limit imposed by fixed overhead.

Further reading

- Denoising Diffusion Probabilistic Models

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 83 - Lecture 83: Video generation: context parallelism and attention optimization

Session: 90 minutes plus an implementation assignment. Preparation: Day 17, Day 78, Day 82.

Learning objectives

- Model spatiotemporal token growth.
- Assess partitioning and temporal quality.

Lecture notes

Video models add a time dimension to spatial representations. Depending on architecture, attention may be joint, factorized, local, or otherwise structured, so derive memory from the actual tensor shapes rather than assuming one universal complexity. Increasing frames and resolution can multiply token count quickly. Decoding and encoding video files also contribute to end-to-end latency.

Context partitioning distributes sequence work but introduces communication and layout constraints. Distinguish memory enablement from speedup, as in tensor parallelism. Evaluate temporal coherence, motion, and frame consistency in addition to individual-frame appearance. A shorter or lower-resolution clip is a changed workload, not a direct acceleration of the original task.

Board example

In a toy joint-attention model with 16 frames and 256 tokens/frame, sequence length is 4,096. Doubling frames to 32 doubles tokens and quadruples dense attention score elements. A factorized temporal/spatial model may scale differently, so inspect its design.

Lab procedure

Create a token/memory estimator for frame count, spatial tokens, and dtype. If a video model is available, measure two supported clip lengths and record preprocessing, denoising, decoding, and file-encoding stages separately.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit predicted versus observed scaling and a temporal-quality rubric.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which tensor or communication becomes the limiting resource.

Further reading

- Video generation guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 84 - Lecture 84: Multi-modal batching: mixing text and image requests

Session: 90 minutes plus an implementation assignment. Preparation: Day 50, Day 58, Day 78.

Learning objectives

- Estimate multimodal request work.
- Preserve fairness across request types.

Lecture notes

A mixed workload may combine text-only requests, one-image requests, and requests with multiple high-resolution images. Scheduling solely by request count can place radically different workloads in the same batch. Use processor-derived visual token counts and text lengths where available, along with modality-specific encoder limits.

Separate encoding and language-model work in the scheduling model. Bucketing can improve padding efficiency but delay uncommon request types. Preserve a fairness or maximum-wait policy so efficiency does not starve large visual jobs. The shared language decoder still needs consistent masks and positions for each modality's input representation.

Board example

A text request has 100 tokens while an image request adds 900 visual tokens to the same 100-token prompt. Treating both as one equal-size admission understates the second's prefill footprint by a factor of ten in this simplified token model.

Lab procedure

Extend Day 50's simulator with text and image-encoder stages, different token budgets, and length bucketing. Compare count-based admission against cost-based admission under the same arrival trace. Include a rare large-image class.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit padding waste, per-class p95 latency, and maximum wait.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain a case where better average utilization worsens one class's experience.

Further reading

- Image-text-to-text inference

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 85 - Lecture 85: Embedding similarity search pipeline: embed → index → query

Session: 90 minutes plus an implementation assignment. Preparation: Day 03, Day 79.

Learning objectives

- Separate embedding and index errors.
- Evaluate recall against an exact baseline.

Lecture notes

A retrieval pipeline chunks documents, embeds them, builds an index, embeds a query, and retrieves candidates. Store the embedding model revision, normalization rule, chunking policy, and document IDs with the index. Query and document representations must follow the checkpoint's compatible conventions; changing the model generally requires rebuilding or migrating the index.

An exact dot-product or distance search provides a reference for approximate indexing. Measure recall@k relative to that reference and separately measure relevance against human or task labels. Approximate-search recall is not the same as semantic usefulness. Include index build time, memory, query latency, and update behavior in the system evaluation.

Board example

If exact top-5 neighbors are {A,B,C,D,E} and approximate top-5 are {A,C,E,F,G}, recall@5 is 3/5=0.6. This says three exact neighbors were retained; it does not establish that any of the five are relevant to the user's question.

Lab procedure

Build exact search over a small embedded corpus, then add an available approximate index. Sweep its search-effort setting, and test a document update plus a model-version mismatch. Keep stable document IDs through the pipeline.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit latency/recall curves and a relevance sample.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how stale index metadata can produce plausible but invalid comparisons.

Further reading

- Faiss getting started

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 86 - Lecture 86: Speech-to-speech pipeline: ASR → LLM → TTS end-to-end latency

Session: 90 minutes plus an implementation assignment. Preparation: Day 68, Day 80, Day 81.

Learning objectives

- Trace first audible response.
- Manage partial transcripts and cancellation.

Lecture notes

A speech pipeline combines endpoint detection, ASR, language-model generation, TTS, and playback. These stages can overlap only when their interfaces accept incremental results. A partial transcript may change, so sending every unstable fragment directly to generation can create repeated or contradictory work. Define when text becomes committed for downstream use.

Measure latency from the user's end of speech to first audible response, along with full-turn duration. Endpoint detection and buffering often dominate perceived delay. Interruption should cancel obsolete generation and queued audio consistently; otherwise the system may continue speaking after a new turn begins. Keep transcript quality and response quality in the same evaluation.

Board example

If endpoint detection takes 300 ms, final ASR 150 ms, LLM TTFT 250 ms, TTS first audio 200 ms, and playback buffering 100 ms sequentially, first response is 1,000 ms after speech ends. Overlap can reduce this only where dependencies allow it.

Lab procedure

Connect mock incremental stages first, recording timestamps and turn IDs. Inject a revised transcript and a user interruption. With available models, replace one mock at a time and measure the changed critical path.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a turn timeline, cancellation trace, and quality examples.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which stage you would optimize first and why.

Further reading

- Whisper reference implementation
- Text-to-speech task guide

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 87 - Lecture 87: Long context: RoPE scaling, context parallelism across GPUs

Session: 90 minutes plus an implementation assignment. Preparation: Day 04, Day 17, Day 34.

Learning objectives

- Distinguish position extrapolation from memory capacity.
- Evaluate long-context quality.

Lecture notes

Rotary position embeddings rotate query/key coordinate pairs according to position. Their relative-position behavior does not guarantee reliable inference at arbitrary lengths. Context extension methods change positional scaling or training behavior and must be evaluated for both short- and long-context tasks. A larger configured maximum length is not evidence of preserved quality.

Memory and compute remain constraints even if position encoding accepts the index. KV storage grows with context, and attention work can grow substantially. Context parallelism distributes sequence work but requires correct global attention semantics and communication. Chunked prefill, covered later, changes scheduling without automatically partitioning total cache capacity across devices.

Board example

For 32 layers, 8 KV heads, dimension 128, FP16, and batch 1, KV storage is 131,072 bytes per token. At 32,768 tokens this is 4 GiB; at 131,072 tokens it is 16 GiB, before weights and workspace.

Lab procedure

Compute capacity for a chosen model, then test retrieval of facts placed at several positions and distances within supported contexts. Compare a documented extension configuration only if available, retaining short-context controls.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit memory estimates and quality by position/length.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why passing a single needle-retrieval example does not establish general long-context reasoning.

Further reading

- RoFormer: rotary position embeddings

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 88 - Lecture 88: EAGLE speculative decoding: feature-level draft vs token-level

Session: 90 minutes plus an implementation assignment. Preparation: Day 15, Day 38, Day 73.

Learning objectives

- Explain feature-based proposals.
- Evaluate a compatible draft mechanism.

Lecture notes

EAGLE explores drafting from target-model features rather than treating the draft as only an independent smaller language model. Its proposal mechanism must be paired with compatible target features, token conditioning, and verification. A generic small model is not an interchangeable EAGLE adapter. Pin the method version because variants change implementation details.

Separate the proposal mechanism from the acceptance procedure. Preserving the target sampling distribution requires the appropriate verification and correction, not merely high feature prediction accuracy. Evaluate end-to-end time, committed progress, and extra memory on a fixed target and workload. Learned adapters introduce their own training and compatibility costs.

Board example

Suppose feature drafting costs 3 ms and verification 9 ms, with an average of three committed tokens per cycle. Effective cost is 4 ms/token. Against a 6 ms/token target-only baseline, speedup is 1.5×. At two committed tokens, the same configuration only breaks even.

Lab procedure

Read the original method and draw the feature/token dependencies. If a compatible supported checkpoint is available, benchmark it against target-only decoding; otherwise simulate the cost model across committed lengths and label the result analytical.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the dependency diagram, compatibility assumptions, and speedup curve.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why feature prediction error and accepted-token progress are different metrics.

Further reading

- EAGLE paper

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 89 - Lecture 89: Medusa: multi-head speculative decoding, measure speedup

Session: 90 minutes plus an implementation assignment. Preparation: Day 15, Day 38, Day 73.

Learning objectives

- Explain multi-head proposals and tree verification.
- Separate greedy and stochastic claims.

Lecture notes

Medusa adds prediction heads that propose multiple future tokens from a shared model representation. Candidate combinations form a tree that can be verified with an attention arrangement preserving each path's context. Heads and verification settings must match the base model. More candidates increase potential accepted progress but also verification work and memory.

Acceptance policies differ in their guarantees. A greedy path check and a typical-acceptance heuristic should not be described as exact target-distribution sampling without proof for the chosen policy. Record which variant is used and evaluate quality accordingly. Include target-only execution and the extra heads' overhead in the comparison.

Board example

If a candidate tree has 20 nodes but only four tokens on its chosen accepted path, the verifier processed 20 candidate positions for four committed tokens. Increasing to 40 nodes is beneficial only if additional progress or parallel efficiency compensates for the added work.

Lab procedure

Draw a depth-three proposal tree and its causal visibility rules. Simulate pruning and path selection. With a compatible model and implementation, measure verification time, candidate count, committed tokens, quality, and memory.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the tree, acceptance-policy statement, and cost/progress comparison.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why equal draft depth does not imply equal verification cost.

Further reading

- Medusa paper

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 90 - Lecture 90: MoE routing from scratch: top-K gating, load balancing loss

Session: 90 minutes plus an implementation assignment. Preparation: Day 04, Day 17.

Learning objectives

- Implement top-k dispatch and combination.
- Quantify expert imbalance.

Lecture notes

A router scores experts for each token, selects a subset, and combines their outputs using gate weights. Define whether selected weights are renormalized and how ties are handled. The dense reference computes selected expert outputs explicitly and provides a correctness target before optimizing dispatch.

Sparse activation reduces the number of experts used per token, but all expert weights still need storage or an offloading plan. Capacity constraints can force dropping, rerouting, or delaying tokens; each changes behavior. A load-balancing training loss encourages distribution but does not guarantee uniform inference traffic, especially under domain-specific prompts.

Board example

For router probabilities [0.6,0.3,0.1] and top-2 selection, renormalized gates are [2/3,1/3]. If the two expert outputs are 3 and 9, the combined scalar output is 5. Without renormalization the result would be 4.5, a different convention.

Lab procedure

Implement a toy router and three linear experts. Test top-1 and top-2, ties, and capacity overflow. Compare vectorized dispatch with an explicit per-token reference and plot assignments per expert for skewed inputs.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit gate/output checks and an imbalance metric.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain the chosen overflow policy and its effect on output equivalence.

Further reading

- Switch Transformers paper

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 91 - Lecture 91: Expert parallelism: simulate routing across N expert shards

Session: 90 minutes plus an implementation assignment. Preparation: Day 41, Day 90.

Learning objectives

- Route tokens to expert owners.
- Model all-to-all communication and stragglers.

Lecture notes

Expert parallelism places different experts on different workers. Token representations travel to the selected expert owners, are processed, and return for weighted combination. Preserve token identity and gate weights across packing, exchange, and reordering; a fast implementation with permuted outputs is incorrect.

Communication volume depends on selected experts, hidden width, dtype, and locality. Worker completion is constrained by the busiest expert shard, not average work. Expert replication or placement can improve balance but consumes additional memory and routing complexity. A simulator should expose these tradeoffs without claiming hardware speed from Python timing.

Board example

If three workers receive 10, 10, and 40 equal-cost expert assignments, the compute makespan is proportional to 40 rather than the mean 20. Even before communication, utilization under a synchronized step is $60/(3\times40)=50\%$.

Lab procedure

Extend Day 90 with expert-to-worker placement, packetized token IDs, configurable network cost, and gather/reorder. Compare balanced versus skewed routing and replicate one hot expert under a fixed memory budget.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit correctness against the single-worker reference and a makespan breakdown.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain when communication outweighs the benefit of additional expert workers.

Further reading

- Switch Transformers paper
- PyTorch distributed execution

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 92 - Lecture 92: Dynamic disaggregation with NVIDIA Dynamo

Session: 90 minutes plus an implementation assignment. Preparation: Day 49, Day 55, Day 91.

Learning objectives

- Model worker-role transitions.
- Account for migration and warmup costs.

Lecture notes

Dynamic allocation changes the balance between prefill and decode resources as workload composition changes. The controller must observe queue pressure, estimate capacity, and account for transition time. A worker cannot be reassigned as though active requests and model state disappear instantly. Draining, loading, cache movement, and readiness belong in the state machine.

Use hysteresis or minimum residence time to avoid repeatedly paying transition cost in response to noisy demand. Backend capabilities determine which transitions are supported; consult the pinned Dynamo release before implementing a real controller. A static split is the baseline, and a zero-cost transition simulator is only an optimistic bound.

Board example

Moving a worker costs 30 seconds, and the new role saves 2 seconds of aggregate waiting per second during a sustained phase. A simple payback estimate is 15 seconds of useful operation after transition, but if the workload switches back immediately, the migration produces no useful return.

Lab procedure

Simulate alternating prompt-heavy and decode-heavy intervals with nonzero transition delay. Compare fixed allocation, instantaneous reallocation, and a draining-aware controller. Track active, draining, loading, and ready worker states.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit queue curves, role timelines, and transition cost.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how the controller prevents capacity from being counted in both roles at once.

Further reading

- Dynamo documentation and release selection

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 93 - Lecture 93: Cache-aware routing: route requests to maximize KV cache hits

Session: 90 minutes plus an implementation assignment. Preparation: Day 16, Day 40, Day 57, Day 74.

Learning objectives

- Optimize completion time rather than hits alone.
- Test stale routing estimates.

Lecture notes

Cache locality can reduce prefill work, but routing also changes queues and future cache contents. Score eligible workers using estimated waiting, uncached prefill work, decode load, and transfer costs where applicable. Use model and tenant compatibility as eligibility rules before optimization. A high hit rate is a means to lower cost or latency, not the sole objective.

Routing feedback can concentrate traffic on one popular-prefix worker. Include queue pressure and consider bounded replication or spillover. Test the policy under stale cache and load information, because perfect-state simulations overestimate routing quality. Eviction can invalidate a previously advertised hit between decision and admission.

Board example

Worker A offers a 50 ms cache saving with 100 ms queue wait; B offers no saving and 20 ms queue wait. Relative to uncached compute, A adds 50 ms and B adds 20, so B is preferable by 30 ms despite its miss.

Lab procedure

Replay mixed repeated and unique prompts using Day 40's cache and Day 57's router. Compare round-robin, hit-maximizing, and queue-aware policies. Inject delayed state updates and measure both hit fractions and completion latency.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit policy tradeoffs and one adversarial trace.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how fallback works when the advertised cache entry has been evicted.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 94 - Lecture 94: Long context without context parallelism: chunked prefill

Session: 90 minutes plus an implementation assignment. Preparation: Day 05, Day 50, Day 87.

Learning objectives

- Schedule long prompts incrementally.
- Distinguish temporary work from total KV capacity.

Lecture notes

Chunked prefill divides prompt processing across scheduling iterations, allowing decode work or other requests to run between chunks. Each chunk must attend to the appropriate preceding context and update positions and cache state consistently. It is a scheduling and workspace technique, not permission to drop parts of the prompt.

Smaller chunks can reduce decode stalls but add scheduling overhead or reduce compute efficiency. They do not by themselves reduce the final full-context KV requirement. Compare output equivalence with an unchunked reference and evaluate both the long prompt's TTFT and other users' inter-token gaps. A mean throughput number can hide either regression.

Board example

A 4,096-token prompt split into 512-token chunks requires eight prefill admissions. If each chunk creates a 10 ms uninterruptible interval, it may replace one larger blocking interval with eight smaller ones, but total work and scheduling overhead must still be measured.

Lab procedure

Extend Day 50 with prompt chunks and decode-priority scheduling. Sweep chunk size and token budget using one long prompt mixed with short interactive jobs. If supported, repeat with a real runtime and verify outputs.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit TTFT, decode-gap, throughput, and peak-memory curves.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain why chunking can improve responsiveness without making a model fit less KV memory.

Further reading

- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 95 - Lecture 95: Fine-tuning a small model for inference quality vs a large quantized one

Session: 90 minutes plus an implementation assignment. Preparation: Day 37, Day 61, Day 72.

Learning objectives

- Define a fair task-specific comparison.
- Include adaptation and serving costs.

Lecture notes

Compare models on a concrete use case with fixed acceptance criteria. A small adapted model may specialize well, while a larger quantized model may retain broader capability. Neither parameter count nor quantization bit width alone predicts task performance. Keep training, validation, and held-out evaluation data separate, and freeze the decision rule before final evaluation.

Measure latency, useful throughput, memory, and quality using each model's appropriate tokenizer and documented prompt format. Token counts may not be directly comparable across tokenizers, so also report request-level outcomes and end-to-end latency. Training cost is amortized over future usage; include it when claiming economic advantage.

Board example

If adaptation costs a hypothetical $100 and saves $0.001 per accepted request, simple break-even is 100,000 accepted requests, excluding maintenance. If the adapted model fails the quality floor, that cost calculation does not make it eligible.

Lab procedure

Choose a narrow classification or extraction fixture. Compare an unadapted small model, a small adapted model if training is available, and a supported larger quantized model. Use a mock cost analysis when training cannot be performed.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a quality/latency/memory table and amortization assumptions.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain one out-of-domain test that could reveal excessive specialization.

Further reading

- Parameter-efficient fine-tuning

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 96 - Lecture 96: Distillation for inference: teacher-student latency/quality tradeoffs

Session: 90 minutes plus an implementation assignment. Preparation: Day 31, Day 95.

Learning objectives

- Distinguish soft-target and sequence distillation.
- Evaluate student quality and speed.

Lecture notes

Distillation transfers behavior from a teacher to a student through training targets. Soft-target distillation can compare probability distributions over a compatible output space; sequence distillation trains on teacher-generated outputs. Different tokenizers complicate token-level logit matching, so do not assume arbitrary teacher and student vocabularies align.

A common classification objective mixes label cross-entropy with temperature-scaled KL divergence between teacher and student distributions. The temperature changes probability softness, and the loss weighting determines how much the student follows labels versus teacher behavior. Generated examples can contain errors; reserve independent evaluation data and inspect failure patterns rather than treating the teacher as ground truth.

Board example

For matching two-class probabilities p=[0.8,0.2] and q=[0.6,0.4], KL(p||q)=0.8 ln(0.8/0.6)+0.2 ln(0.2/0.4)≈0.0915 nats. Matching only the argmax discards this confidence information.

Lab procedure

Implement a small classifier distillation experiment or a sequence-target dataset pipeline. Compare the same student with and without distillation, using identical held-out examples. Measure student inference latency separately from teacher data-generation and training costs.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the target-generation method, loss definition, quality changes, and deployment cost model.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how teacher mistakes can propagate into the student.

Further reading

- Distilling the Knowledge in a Neural Network

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 97 - Lecture 97: Intelligence evaluation: build an eval harness for a deployed model

Session: 90 minutes plus an implementation assignment. Preparation: Day 37, Day 47, Day 77.

Learning objectives

- Build task-grounded evaluation cases.
- Distinguish quality regressions from harness failures.

Lecture notes

An evaluation harness needs a task definition, representative inputs, expected behavior or scoring rubric, and a stable execution record. Include normal, boundary, and failure cases. Version prompts, model settings, and fixtures so a score change can be attributed. Structural validity, factual correctness, and user usefulness may require separate scores.

Use deterministic checks where possible and blinded human or model-assisted rubrics where necessary. A model judge is itself fallible and should be calibrated against reviewed examples. Keep timeouts and malformed responses in the report. Repeated stochastic trials and confidence intervals help distinguish small apparent gains from noise; avoid tuning repeatedly on the final held-out set.

Board example

Model A passes 90 of 100 cases and B passes 92. The two-point difference alone does not establish a reliable improvement. Inspect paired outcomes: which cases changed, whether failures cluster, and how repeated trials affect the comparison.

Lab procedure

Build a 30-case fixture spanning at least three categories with explicit scoring. Run a reference and candidate endpoint or mock adapters. Record raw responses, parse failures, rubric scores, latency, and exact configuration.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit a per-category comparison and release recommendation.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how you would detect a harness bug that mistakenly treats missing outputs as passes.

Further reading

- Language Model Evaluation Harness

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 98 - Lecture 98: Design: sketch the full inference stack for a real use case

Session: 90 minutes plus an implementation assignment. Preparation: Day 61, Day 77, Day 97.

Learning objectives

- Translate a use case into measurable requirements.
- Select a defensible serving design.

Lecture notes

Choose one concrete application such as document extraction, a local assistant, or embedding retrieval. Define workload distributions, quality acceptance, latency targets, maximum concurrency, and available hardware. Scope the first version tightly enough to measure. A design is incomplete if it lists components without explaining why their capacities and interfaces fit the workload.

Draw the request path from client through admission, serving, and observation. Estimate weight, KV, and workspace memory, then forecast capacity using measured baselines or clearly labeled assumptions. Define overload, timeout, cancellation, and rollback behavior. Identify the largest uncertainty and design a small experiment to resolve it before adding complexity.

Board example

If expected arrivals are 5 requests/s and mean time in system is 2 seconds, stable mean concurrency is 10. A design targeting only two concurrent requests cannot satisfy that workload without changing service time, queueing, or rejection behavior.

Lab procedure

Write a two-page design using results from earlier lectures. Include a component diagram, capacity worksheet, model/runtime manifest, three failure scenarios, and an acceptance table with numeric targets.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the design and a prioritized implementation plan.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain which choice you would revisit first if prompt lengths doubled.

Further reading

- Google SRE release engineering

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 99 - Lecture 99: Build: FastAPI + vLLM inference server with health checks and metrics

Session: 90 minutes plus an implementation assignment. Preparation: Day 53, Day 63, Day 68, Day 98.

Learning objectives

- Build a minimal request path.
- Expose readiness, metrics, and bounded admission.

Lecture notes

Implement the designed service using a small API layer and a supported inference runtime. Keep model execution ownership clear: an API wrapper should not accidentally load a second model copy or block its event loop with synchronous heavy work. Validate input limits and map runtime failures into a documented response contract.

Add readiness, liveness, bounded admission, timeouts, and streaming cancellation as part of the request lifecycle. Metrics should reconcile arrivals, terminal outcomes, and in-flight work. Use a mock runtime adapter to test failure behavior without requiring expensive model execution. A successful endpoint demo does not establish load capacity or rollout safety yet.

Board example

With a queue limit of 20 and two active slots, the twenty-third simultaneous request cannot be silently accepted into an unbounded internal queue. Define whether it receives a retryable rejection or another documented response, and include it in metrics.

Lab procedure

Build the API and runtime adapter, then exercise success, oversized input, queue saturation, timeout, cancellation, and unavailable-model cases. Run a small inference smoke fixture when the supported runtime is available.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit source, launch instructions, API examples, and lifecycle test results.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain how disconnecting a streaming client releases server-side work.

Further reading

- FastAPI asynchronous execution
- vLLM documentation

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 100 - Lecture 100: Deploy: ship it to the home lab cluster with load balancing

Session: 90 minutes plus an implementation assignment. Preparation: Day 57, Day 62, Day 99.

Learning objectives

- Deploy reproducible artifacts.
- Demonstrate routing and recovery.

Lecture notes

Deploy the capstone into the available lab with explicit model, image, and configuration versions. Keep service discovery and routing simple enough to inspect. Readiness should prevent traffic reaching unloaded workers, and the load balancer should stop new admissions to unhealthy ones while accounting for existing streams. Document the hardware topology and resource allocation actually used.

Exercise recovery rather than inferring it from a successful startup. Restart a worker, interrupt a stream, and roll back a version in a controlled lab. Verify whether requests fail, retry, or finish on their original workers, and report the user-visible result. A single-host setup can demonstrate lifecycle behavior but cannot prove multi-host fault tolerance.

Board example

Two workers each sustain 4 requests/s and serve a total 6 requests/s. Losing one leaves a deficit of 2 requests/s. Over a 15-second restart, about 30 requests accumulate unless admission rejects excess work or spare capacity is available.

Lab procedure

Deploy the Day 99 artifacts, run the acceptance smoke suite, and execute one worker restart plus one rollback. Collect readiness events, routing decisions, queue depth, error counts, and model-version responses.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the deployment manifest, recovery timeline, and observed limitations.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain whether the design meets its target under one-worker failure.

Further reading

- Docker build practices
- Google SRE release engineering

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 101 - Lecture 101: Optimize: run the benchmark harness, find the bottleneck, fix it

Session: 90 minutes plus an implementation assignment. Preparation: Day 52, Day 71, Day 77, Day 100.

Learning objectives

- Choose a bottleneck from evidence.
- Validate a single controlled improvement.

Lecture notes

Use the benchmark harness and traces to identify the dominant constraint under the capstone workload. Optimize one cause at a time: scheduling, prefix reuse, quantization, kernel execution, or client behavior. State the expected effect before running the experiment. Keep the original baseline and preserve quality, workload, and resource accounting across comparisons.

Evaluate the whole service after a local improvement. A faster kernel may move the bottleneck to tokenization or networking, and a larger batch may improve throughput while violating latency. Repeat measurements and inspect failures. Reject a change that improves one headline metric by changing the task or hiding bad outcomes.

Board example

If model execution accounts for 60% of end-to-end time and an optimization halves it, the ideal overall speedup is $1/(0.4+0.6/2)=1.43\times$. A claimed $2\times$ end-to-end gain needs an additional explanation beyond that kernel improvement.

Lab procedure

Select one bottleneck, predict its maximum benefit, implement one change, and rerun the frozen acceptance workload. Compare quality, p95 TTFT, throughput, peak memory, cost model, and failures with the baseline.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the hypothesis, patch, raw results, and retain-or-revert decision.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain the next bottleneck and whether another optimization is worthwhile.

Further reading

- PyTorch benchmarking

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.

Day 102 - Lecture 102: Reflect: what I learned, what I'd do differently, what's next

Session: 90 minutes plus an implementation assignment. Preparation: Day 97, Day 98, Day 101.

Learning objectives

- Present reproducible evidence.
- Distinguish demonstrated results from future work.

Lecture notes

A final report should make the system understandable and its claims auditable. Restate the use case and targets, describe the implemented design, and present baseline versus final results under the same workload. Include failed experiments when they explain a tradeoff or prevent repeated mistakes. Separate measured outcomes, analytical estimates, and unimplemented ideas.

Reproducibility requires artifact versions, environment details, fixtures, commands, and result files. A reader should be able to locate the evidence behind each claim. Discuss limitations such as small evaluation sets, unavailable hardware, simulated failure behavior, or narrow prompt distributions. Use these limits to prioritize the next experiment rather than promising unsupported generality.

Board example

If throughput improves from 100 to 140 tokens/s while p95 TTFT rises from 80 to 130 ms against a 100 ms target, the final configuration fails that target. A balanced defense reports the 40% throughput gain and the latency regression, then chooses a compliant operating point.

Lab procedure

Prepare a ten-minute defense with problem, architecture, experiment design, results, failures, and next steps. Re-run the documented smoke path from a clean environment or have a peer follow it. Archive the exact manifests and evaluation outputs.

Use the measurement and submission conventions. This is a student exercise specification; reference implementations and recorded GPU measurements are not bundled for this day.

Problem set and deliverable

Submit the final report, reproduction guide, and evidence index.

1. Reproduce the board calculation and state its units and assumptions.
2. Complete the lab comparison, keeping its baseline and raw observations.
3. Describe one failure case and how your implementation or analysis handles it.

Check your understanding

Explain the strongest conclusion supported by your experiments and one attractive claim the evidence does not support.

Further reading

- Google SRE release engineering

For runtime APIs and hardware support, use documentation matching the version in your experiment manifest. The numerical example is illustrative, not a reported benchmark.